

ZLang HDL

User Guide

Community Release

v0.1.0a24

Installation, projects, simulation, debugging, waveforms and RTL workflows

Apache-2.0 Community distribution

Contents

1 Two complete workflows . . . 4
1.1 Install, simulate Counter and open a saved waveform . . . 4
1.2 RuleAction: F9, Continue, highlighted source and live signals . . . 4
1.3 Where stimulus comes from . . . 4
2 Getting started . . . 5
2.1 Install for development . . . 5
2.2 A first module . . . 5
2.3 Check named verification goals . . . 6
2.4 Source files . . . 6
2.5 Reading common notation . . . 7
2.6 Where to continue . . . 7
3 Installation, WSL2, and external tools . . . 8
3.1 Recommended installation with uv . . . 8
3.2 Conventional venv and pip alternative . . . 10
3.3 Windows through WSL2 . . . 10
3.4 Check and run ZLang . . . 11
3.5 Choose which external tools you need . . . 11
3.6 Recommended: OSS CAD Suite . . . 12
3.7 Alternative: install components separately . . . 12
3.8 Verify the installation . . . 12
3.9 Troubleshoot formal runs . . . 13
4 Projects and dependencies . . . 14
4.1 Project manifest . . . 14
4.2 Lock update . . . 14
4.3 Optional exact compiler and stdlib lock . . . 15
4.4 Offline compilation . . . 16
4.5 Identity and artifacts . . . 16
4.6 Deliberate boundaries . . . 16
5 Native simulation . . . 17
5.1 Four-state initialization and diagnostics . . . 17
5.2 FST/VCD waveforms and differential RTL comparison . . . 18
5.3 Debug without an event file . . . 19
5.4 Python simulation without an event file . . . 19
5.5 Simulation-only typed file I/O . . . 19
6 Debugging native simulation in VS Code . . . 20
6.1 Prerequisites . . . 20
6.2 Quick start . . . 20
6.3 JSONL event files . . . 21
6.4 Debug without an event file . . . 22
6.5 Python simulation without an event file . . . 22
6.6 Breakpoints, stepping, and values . . . 23
6.7 Live and saved waveforms . . . 23
6.8 Project launches . . . 24
6.9 Troubleshooting . . . 25
6.10 Architecture and contracts . . . 25
6.11 Initial supported surface . . . 25
6.12 Waveforms . . . 26
6.13 Packaging boundary . . . 26
6.14 Source simulation channels . . . 26
7 Waveform workspace . . . 27
7.1 Refresh a simulation result . . . 27
7.2 Open and arrange signals . . . 27
7.3 Numeric analog display . . . 28
7.4 Navigate the trace . . . 28
7.5 Values, bookmarks, and measurements . . . 29
7.6 Portable sessions . . . 29
7.7 Live debugger traces . . . 29
7.8 Troubleshooting . . . 30

7.9 Waveform Control Protocol . . . 30
8 Generate SystemVerilog . . . 32
8.1 Connect to vendor FPGA debug tools . . . 32
9 Implementation profiles . . . 33
9.1 Semantic regions . . . 33
9.2 Normalization and conflicts . . . 33
9.3 Reading the backend selection report . . . 34
10 Target platform descriptions . . . 35
10.1 Compiler-shipped source descriptions . . . 35
10.2 Manual selection . . . 35
10.3 The selected graph . . . 35
10.4 Backend and manifest . . . 36
10.5 Boundaries . . . 36
11 Target resources . . . 37
11.1 Implemented model . . . 37
11.2 DSP48E1 pipeline binding . . . 37
11.3 Signed-product cascade . . . 37
11.4 RAMB36 validation . . . 37
11.5 Intel and ASIC status . . . 38
11.6 Clock-resource blocker . . . 38
11.7 Implementation artifact contents . . . 38
12 Target-aware architecture and pipeline planning . . . 40
12.1 User contract . . . 40
12.2 Candidate generation and legality . . . 40
12.3 Timing DAG . . . 41
12.4 How target evidence affects selection . . . 41
12.5 Current boundary . . . 41
13 Platform constraints . . . 42
13.1 Clock constraints . . . 42
14 Direct-SV . . . 43
14.1 Supported composition subset . . . 43
14.2 Simulation-only architectural state access . . . 44
14.3 Formal boundary . . . 45
14.4 Explicitly unsupported . . . 45
14.5 Example and structural coverage . . . 46
15 Formal verification . . . 47
16 Whole-build manifests and evidence . . . 48
16.1 Compiler and implementation identities . . . 48
16.2 Published files and companions . . . 48
16.3 Exact evidence meanings . . . 49
16.4 Harness generation is not proof execution . . . 49
16.5 Counterexample and source attribution . . . 50
16.6 CLI publication . . . 50
16.7 Current limitations . . . 51
17 Structured diagnostics . . . 52
18 Generated source maps . . . 53
18.1 Exactness boundary . . . 53
18.2 Generated-RTL navigation boundary . . . 53
19 Generated navigation bundles . . . 54
19.1 Why generated navigation uses a separate bundle . . . 54
19.2 Layout and schema . . . 54
19.3 Publication and validation . . . 55
19.4 Source freshness . . . 55
19.5 Relocatability and limitations . . . 55
20 Compiler tooling API . . . 56
20.1 Request-scoped analysis . . . 56
20.2 Diagnostic edits . . . 56
20.3 Generated RTL artifacts and navigation limits . . . 56
20.4 Document symbols . . . 57
20.5 Semantic hover . . . 57
20.6 Definitions . . . 57

20.7 References . . . 57
20.8 Rename . . . 58
20.9 Completion . . . 58
20.10 Signature help . . . 58
20.11 Semantic tokens . . . 59
20.12 Reusable tooling sessions . . . 59
21 Language Server Protocol and VS Code . . . 60
21.1 Supported now . . . 60
21.2 Cache configuration . . . 62
21.3 Not implemented . . . 63
Font license notice . . . 64

Installation, projects, simulation, debugging, waveforms and RTL workflows.

1 Two complete workflows

These routes use a matching a24 compiler, native runtime and Linux-x64 VSIX. The universal VSIX supplies editor/LSP support, not the native debugger. See installation for wheel-only and WSL2 alternatives.

1.1 Install, simulate Counter and open a saved waveform

From the root of a Git checkout:

```
make install
source .venv/bin/activate
zlang examples/counter.zhl --top Counter --check
zlang sim examples/counter.zhl --top Counter --events examples/counter.events.jsonl --
    trace build/counter.fst
```

Open `build/counter.fst` in VS Code with the ZLang HDL waveform editor. Add `Counter.clk`, `Counter.rst` and `Counter.y` from the Design Browser. Select a row before using Change to navigate that signal's transitions. The checked-in schedule asserts reset for one event, then releases it and increments twenty times: `y` ends at 20. The bounded clock/cycle alternative does not invent a reset sequence; use JSONL for explicit reset/input stimulus. See native simulation and saved/live waveform controls.

Native trace time is **event-domain ticks**, two ticks per simulator event. It is not measured hardware nanoseconds and does not imply a physical frequency. External VCD/FST retains its genuine declared timescale.

1.2 RuleAction: F9, Continue, highlighted source and live signals

1. Open the checkout folder in VS Code; run `make install` and reload the window after installing the matching Linux-x64 extension.
2. Open `examples/rule_action.zhl`. Put the caret on the executable `count <-` rule line and press **F9**. A red dot means a configured breakpoint, not that execution is stopped there.
3. Choose the checked-in RuleAction launch configuration and press **F5**. It uses the saved event schedule.
4. Press **Continue**. When the breakpoint is reached, VS Code highlights the current source statement and shows the paused stack/variables.
5. Select a value in Variables and invoke **Add to Waveform**. Step and observe the distinct LIVE marker. Moving waveform cursors does not run the simulator.

The debugger guide contains the complete `launch.json`, event schedule, supported types, reset behavior and limitations.

1.3 Where stimulus comes from

JSONL is newline-delimited JSON authored by the user or emitted by a test-vector generator. Each event sets inputs/resets and advances named clock edges. It is shared by the CLI, native debugger and native-versus-RTL comparison. For bounded interactive exploration, a `clock/cycles` launch can run without an event file; while paused, change supported input values. Reset is still explicit.

A Python testbench can drive the native simulation API directly and check values without writing JSONL. This is a separate Python workflow: Python breakpoints do not share one stepping engine with `.zhl` breakpoints in the native debugger. Export JSONL from Python when the same stimulus must be replayed under that debugger.

2 Getting started

The representative executable language tour is `examples/all_syntax.zhl`. It is a practical starting point rather than an exhaustive list of every legal composition. See Known limitations for deliberately unsupported forms.

Physical ZLang HDL source files use the canonical `.zhl` suffix and MIME type `text/x-zlang-hdl`. Logical imports remain extension-independent. The former `.zl` spelling is intentionally rejected so these sources cannot be confused with the unrelated language that already owns that extension.

2.1 Install for development

This alpha release requires CPython `>=3.12,<3.13`. The recommended `uv` workflow does not require a suitable Python interpreter to be installed first; `uv` can download and manage the verified runtime. From the repository root:

```
make install
source .venv/bin/activate
```

This is the ordinary compiler/native/LSP setup. Compiler contributors can use `make venv PYTHON_PROFILE=dev` for test/formal dependencies and Ruff; the default `make venv` selects the complete release-maintainer profile.

If a compatible interpreter is already available, the complete installation guide also documents a conventional `venv`/pip path without relying on a version-specific executable name. Windows users should run the Linux flow inside WSL2 rather than installing the compiler into native Windows Python.

Checking source and generating SystemVerilog do not require an EDA installation. Verilator, Yosys, SymbiYosys, `yosys-smtbmc`, Z3 and Icarus are optional external programs for the corresponding lint, synthesis, formal and simulation flows. See Installing ZLang HDL and the external EDA toolchain for the recommended OSS CAD Suite route, separate component installation, release-tested versions and a real formal smoke test.

Check a source file without generating output files:

```
zlang examples/all_syntax.zhl --check
```

Without `--top`, `--check` validates every declared module. With `--top NAME`, it validates only that selected root and does not generate RTL or run formal checks.

2.2 A first module

```
module Add {
    in  a : u8
    in  b : u8
    out y : u9

    y = a + b
}
```

Hardware assignments are concurrent. Source order does not turn `=` into software-style sequencing. `=` drives a combinational value; `<-` schedules a register's next value at its clock edge.

Generate Direct-SV and lint it:

```
zlang examples/extended_add.zhl \
  --systemverilog build/ExtendedAdd.sv
verilator --lint-only --top-module ExtendedAdd build/ExtendedAdd.sv
```

An explicit artifact path keeps stdout empty. Use `--verbose` for success messages on stderr. A bare invocation emits the production Direct-SV artifact to stdout; use `--check` for semantic validation without emission.

2.3 Check named verification goals

Clocked modules can carry same-cycle safety and bounded-reachability goals:

```
assert count_within @ clk { count <= DEPTH }
cover reaches_full @ clk { count == DEPTH }
```

Run the applicable verification jobs and source goals:

```
zlang design.zhl --top Top --verify \
  --formal-jobs 4 \
  --verification-report build/verification.txt \
  --verification-work-dir build/verification-work
```

Or publish a hash-validated bundle and replay it without recompiling source:

```
zlang design.zhl --top Top \
  --verification-bundle build/verify
zlang verify build/verify --mode bmc --depth 20 \
  --work-dir build/verify-work
```

`bounded_pass` means only that no counterexample was found through the stated depth. A cover reports `witnessed` or `bounded_unreached`; a cover miss is not a safety failure. A `prove` request runs BMC first and starts proof only when every safety job is `bounded_pass`. Solver configurations, logs, and VCDs are retained in the external work directory; the immutable bundle is not modified. `--formal-jobs` runs independent jobs in parallel while keeping result order deterministic.

Each verification goal is routed against its own declared clock/reset pair. Goals in two supported synchronous domains can execute independently; an unsupported domain skips only goals in that domain. This does not add a cross-domain temporal property.

2.4 Source files

A file may contain imports, type aliases, structs, pure functions, operator overloads, rewrite declarations, target/resource declarations, and one or more modules:

```
import std.math.complex

type Byte = u8

struct Pair<type T> {
    left  : T
    right : T
}

fn widen(x : Byte) -> u9 { extend<9>(x) }

module Example {
    in  x : Byte
    out y : u9 = widen(x)
}
```

Select a top explicitly when a file contains several modules:

```
zlang design.zhl --top Example --check
```

Names are case-sensitive. There are no semicolons. Line comments use `//`. Non-nested block comments use `/* ... */`; both forms are lexical whitespace. Decimal, hexadecimal (`0x`), and binary (`0b`/`0B`) integer literals may contain valid underscore separators.

2.5 Reading common notation

Form	Meaning
`name = expression`	Drive a combinational value or resource control.
`state <- expression`	Schedule next-state at the active edge.
`source -> destination`	Connect typed endpoints or bind a command.
`key => expression`	Select a `switch`/choice arm.
`thing @ clk`	Associate a declaration with a clock domain.
`field @ 7:4`	Place a CSR field.
`object.field`	Select an aggregate, protocol, or resource member.
`values[index]`	Read a range-proven vector element; the selector may be runtime.
`raw[index]` for a packed scalar	Read one LSB-zero packed bit; a runtime selector needs a compiler-proven in-range unsigned interval.
`raw[offset +: WIDTH]`	Read a fixed-width LSB-based packed window; `WIDTH` is compile-time and the runtime offset must be provably in range.

ZLang requires exact canonical types at assignment boundaries. It never silently truncates, extends, performs a numeric signed/unsigned conversion, or rescales fixed-point values. Equal-width raw boundaries involving `bits<N>` or flat `vec<N,bit>` are the documented representation-only exception; use `bitcast<T>` explicitly when the boundary is not already typed.

2.6 Where to continue

- Types and numerics
- Expressions, functions, and generics
- Sequential logic, rules, and storage
- Hierarchy, protocols, and composition
- Formal verification
- SystemVerilog generation and CLI
- Standard library
- Reproducible projects and dependencies
- Named module interfaces
- Runnable examples

3 Installation, WSL2, and external tools

ZLang HDL supports CPython >=3.12,<3.13. The source compiler is release-tested on Linux x86-64. Native-simulation wheels are supplied for Linux x86-64 (including WSL2). macOS native wheels are not part of this release. You do not need to find a distribution package for that exact Python runtime: the recommended uv workflow can download and manage it independently of the system Python. Parsing, semantic checking and Direct-SV generation need only the Python package. Verilator, Yosys, SymbiYosys and a solver are external programs used only when their corresponding lint, synthesis or formal flow is requested.

The exact versions used by release acceptance are recorded in release/status.json. Other versions may work, but they are not evidence for the published release.

3.1 Recommended installation with uv

Install uv using its official platform instructions. Confirm that the executable is visible in the same terminal that will run ZLang:

```
uv --version
```

For ordinary use, install the wheel downloaded from the matching GitHub release as an isolated command-line tool. uv provisions the requested interpreter if it is not already present:

```
uv tool install --python '>=3.12,<3.13' \
  --with /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
zlang lock --version
zlang verify --version
```

If uv reports that its tool directory is not on PATH, run uv tool update-shell, then start a new shell. uv tool dir --bin prints the command directory for manual PATH configuration. To replace an installation made from a downloaded wheel, repeat uv tool install --force ... with the new wheel.

For Python testbenches or an isolated library installation rather than a tool, create a local environment and install both matching wheels into it:

```
uv venv --python '>=3.12,<3.13' .venv
source .venv/bin/activate
uv pip install \
  /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
```

For use from a Git checkout, install these host prerequisites first:

- A glibc-based Linux x86-64 host, including WSL2. The reviewed native runtime wheel requires glibc 2.28 or newer; the Linux-x64 debugger/viewer executables require glibc 2.34 or newer.
- Git, GNU Make, Bash 4 or newer, and standard shell utilities.
- Either uv, which can provision CPython 3.12, or an existing CPython 3.12 with working venv and ensurepip support. First setup needs network access for Python packages and, with managed Python, the interpreter.
- For optional editor installation: VS Code's code CLI, Node.js 22, npm and readelf from binutils. ZLang does not install these host applications.

Without Node.js/npm or the VS Code CLI, compiler/native/LSP setup can complete without changing the editor. INSTALL_EDITOR=0 explicitly selects that route. EDA tools are separate prerequisites for the flows that use them. A normal checkout installation uses reviewed native binaries and needs no Rust or C++ build toolchain.

Keep the environment local to the repository:

```
git clone https://github.com/postoroniy/zlang-hdl.git
cd zlang-hdl
make install
source .venv/bin/activate
zlang --version
```

`make install` installs the checkout compiler, LSP and its matching reviewed native wheel into one local environment, without Python test or release tools. No profile selection is needed. `make venv` retains the complete maintainer setup. On a supported host with VS Code and Node.js 22, it also installs the matching extension. Use `INSTALL_EDITOR=0` to leave the installed extension unchanged. Installing only `.[test]` does not install the native runtime and is not equivalent to this complete setup.

`make -n install` prints the bootstrap recipe without installing anything. A real bootstrap rejects an active sibling environment or an incompatible local interpreter before installation. Deactivate the foreign environment and retry; move an incompatible `.venv` aside explicitly, since bootstrap does not delete it. Stale checkout distribution metadata is preserved below ignored build output rather than allowed to shadow the installed compiler. If optional editor setup fails, Python/native/LSP setup may already be complete: retry `make editor-install`, or use `make install INSTALL_EDITOR=0`. With missing Node.js/npm, install editor prerequisites separately. `KEEP_TMP=1 make install` retains bootstrap failure logs.

To replace an old waveform/editor UI after a pull, run `make editor-install` with Node.js 22 and reload VS Code. Close/reopen waveform tabs: existing webviews keep their loaded JavaScript. Set `VSCODE=/path/to/code` for an alternate executable.

Install the matching extension and set `"zlang.lsp.path"` in VS Code to the absolute path of the environment's `zlang-lsp` executable. In a checkout this is `${workspaceFolder}/.venv/bin/zlang-lsp`. See the language-server guide for diagnostics/navigation and the debugger guide for the Linux-x64 native-debug requirements.

The commands below use `zlang` directly: it is on `PATH` after `uv tool install`, or after activating the checkout's virtual environment with `source .venv/bin/activate` in a Bash/Zsh terminal. Alternatively, in the checkout use `uv run --no-sync zlang ...` without activating it. Open a new terminal and activate the environment again when returning to the checkout.

3.1.1 Checkout Python dependency profiles

For ordinary use, `make install` selects runtime dependencies and refreshes the VS Code extension when prerequisites are available. The existing bootstrap also supports explicit additive profiles for development and release work:

Profile	Python dependencies	Editor default
Runtime	Compiler, native runtime and LSP; no test or release tools	Unchanged
Development	Runtime plus the complete test/formal dependencies and Ruff	Unchanged
Release	Development plus all pinned release, license and dependency-audit tools	Refresh when prerequisites are available

For explicit profile selection:

```
make venv PYTHON_PROFILE=runtime
make venv PYTHON_PROFILE=dev
make venv PYTHON_PROFILE=release
```

All profiles retain the pinned pip/setuptools bootstrap for editable installation. `release` remains the default. No tests, security checks or release gates are removed by choosing a smaller installation. Use the release profile before running release and audit commands; smaller environments

do not bypass missing prerequisites. `INSTALL_EDITOR=0` always leaves the extension unchanged; `INSTALL_EDITOR=1` explicitly enables its existing build/install workflow.

Profiles never uninstall or synchronize packages already in an environment. Selecting `runtime` in an existing development environment will retain its existing packages. For a separate runtime environment below the same checkout:

```
make venv PYTHON_PROFILE=runtime VENV=.venv-runtime
source .venv-runtime/bin/activate
```

Pass that same `VENV=.venv-runtime` to subsequent Make commands. Deactivate it before using another environment. Each environment remains owned by its checkout.

The optional `formal` extra installs the checkout's pinned Python Z3 bindings without pytest or release tooling. After activating a runtime checkout environment:

```
python -m pip install -e '.[formal]'
```

The complete `test` extra already includes the same solver package, so development and release profiles retain it. External Yosys/SymbiYosys/SMT executables still need separate provisioning. Without Python Z3, eligible arithmetic proofs retain the raw SMT route; missing tools, timeouts and inconclusive results do not become proofs. See formal verification.

These commands configure a source checkout. Existing downloaded wheels and editor packages retain their original dependency metadata.

3.2 Conventional venv and pip alternative

If a compatible Python is already available, use whatever executable name the host provides; the example deliberately avoids assuming a version-suffixed command:

```
python3 -c 'import sys; assert (3, 12) <= sys.version_info < (3, 13), sys.version'
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install \
  /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
```

For an editable development checkout, replace the two-wheel installation command with:

```
python -m pip install \
  /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  -e '.[test]'
```

The native wheel must match this checkout's required version and platform. This fallback needs a compatible interpreter and `venv` support from the host. ZLang should be installed with `uv` or an isolated virtual environment, leaving the operating system's own Python unchanged.

3.3 Windows through WSL2

The current release is tested as a Linux application, not as a native Windows Python package. On Windows 10/11, install WSL2 with an Ubuntu distribution, open the Ubuntu shell, and perform the Linux installation entirely inside it. For example:

```
sudo apt-get update
sudo apt-get install -y curl git build-essential
```

Then install `uv` using its official Linux instructions and follow the `uv` steps above. Keep active projects under the WSL Linux filesystem, for example `~/src/zlang-hdl`, rather than a Windows-

mounted drive; this generally gives much better compiler, Git, tests and EDA-tool filesystem performance.

Inside a development checkout, these forms are equivalent:

```
source .venv/bin/activate
zlang examples/add.zhl --check
uv run --no-sync zlang examples/add.zhl --check
```

`uv run --no-sync` uses the already-created project environment without changing dependencies. Scripts should either activate the environment first or use `uv run --no-sync`, making them independent of interactive shell state.

3.4 Check and run ZLang

Check the installed commands:

```
zlang --version
zlang lock --version
zlang verify --version
```

Check a design and emit production Direct-SV without installing any external EDA program:

```
zlang examples/add.zhl --check
mkdir -p build
zlang examples/add.zhl --systemverilog build/Add.sv
```

3.5 Choose which external tools you need

Only a compatible Python runtime and the `zlang-hdl` package are mandatory for checking source and emitting SystemVerilog. `uv` is an optional installer: it can obtain that Python runtime, but ZLang does not need `uv` after installation. Install other tools only for the workflows you actually use:

Workflow	Additional requirement	If it is missing
`zlang SOURCE --check` and SystemVerilog emission	None beyond Python and `zlang-hdl`	Neither Verilator nor Yosys is needed; a missing compiler package or incompatible Python prevents `zlang` from starting.
`zlang sim` (native simulation)	Matching `zlang-native-sim` wheel for the host	Simulation fails explicitly; compiler checking and RTL emission remain available.
Strict RTL lint or Verilator execution	`verilator`	RTL can still be emitted; lint and Verilator simulation cannot run.
Generic synthesis	`yosys`	RTL can still be emitted; synthesis cannot run.
Formal BMC, proof, or cover	`yosys`, `sby`, `yosys-smtbmc`, and one selected solver executable	Checking and RTL emission still work; requested formal jobs cannot complete.
Icarus-based test or simulation flow	`iverilog` and `vvp`	Only that selected flow is unavailable.
`zlang lock update` with Git dependencies	`git` and access to the pinned revision during update	Path-only projects and locked offline compilation still work; fetching a Git dependency does not.

`yosys-smtbmc` is the Yosys SMT model-checking driver. It is the executable sometimes informally shortened to “BMC”; there is no ZLang dependency named `bmct`. The default recorded formal route invokes the external `z3` executable through this driver. Bundle replay may instead select

another supported SMTBMC solver, such as `boolector`, `bitwuzla`, `yices`, or `cvc5`, with `zlang verify BUNDLE --solver NAME`. Installing only a Python solver module is not a substitute for putting the corresponding executable on `PATH`.

3.6 Recommended: OSS CAD Suite

The simplest reproducible route is the official YosysHQ OSS CAD Suite. Its release archive provides Yosys, SymbiYosys, solvers and related open-source EDA tools together. Download the archive for the host platform from the OSS CAD Suite releases, extract it, then either source its environment file for each shell:

```
source /absolute/path/to/oss-cad-suite/environment
```

or add its `bin` directory to `PATH`:

```
export PATH="/absolute/path/to/oss-cad-suite/bin:$PATH"
```

Use an absolute installation path and keep the suite intact because Yosys and SymbiYosys also use adjacent data and support files. See the upstream OSS CAD Suite release page linked above for supported archives and platform details.

Install Verilator separately when it is not supplied by the selected suite. A distribution package is convenient:

```
sudo apt-get update
sudo apt-get install verilator
```

Distribution versions vary. Follow the upstream Verilator installation guide when the release-tested version or a source build is required.

3.7 Alternative: install components separately

Component installation is useful for distribution packaging or when a pinned tool is maintained independently:

- install Yosys using the upstream Yosys build/install instructions;
- install SymbiYosys using the upstream SymbiYosys documentation;
- install the `z3` command from the official Z3 releases or a suitable distribution package;
- install Verilator using its official guide linked above;
- install Icarus Verilog from the host distribution when the selected test or simulation flow requires `iverilog`/`vvp`.

Keep all selected tool `bin` directories on the same `PATH` used to start ZLang, the terminal, CI job or VS Code. Mixed host/container installations are a common reason for a tool being visible interactively but unavailable to the compiler.

3.8 Verify the installation

```
verilator --version
yosys -V
sby --version
command -v yosys-smtbmc
z3 --version
boolector --version
bitwuzla --version
iverilog -V
```

The release-tested external-tool versions are recorded only in the release status under `eda_toolchain`; the supported Python range is recorded under `platform.python`. These are evidence for that release, not universal tool-version requirements. Other versions may work; validate

them for the flow you intend to use. Finding an executable on `PATH` is not, by itself, evidence that a formal or synthesis result is valid.

Run a real formal smoke test:

```
zlang examples/verification/bounded_counter.zhl \
  --verify --verify-require proven \
  --formal-depth 16 --formal-timeout 45 \
  --verification-bundle build/verify/counter \
  --verification-work-dir build/verify-work/counter \
  --verification-report build/verify/counter-result.txt
```

Expected exit status is zero. The `capacity` assertion should be `proven`, the `reaches_capacity` cover should be `witnessed`, and `exceeds_capacity` should be `bounded_unreached`. A bounded cover miss is not a proof of unreachability. See the runnable formal examples for a deliberate counterexample, scoped assumptions, immutable bundle replay and ready/valid checking.

3.9 Troubleshoot formal runs

Formal execution is opt-in. `--check`, normal compilation and SystemVerilog emission do not invoke a solver. `bounded_pass` is bounded evidence and is never reported as `proven`; unavailable tools or bindings remain explicit `unknown`/`skipped` according to the requested policy.

If a tool is not found, run the version commands above in the same environment that starts ZLang. If a proof is `unknown`, inspect the retained report and solver logs under `--verification-work-dir`; increasing depth or timeout does not turn an unsupported binding or reset model into an executable property.

4 Projects and dependencies

ZLang source imports are logical names. Physical paths, Git URLs, and revisions belong to project metadata, never to `.zhl` source:

```
import acme.dsp.filters
import std.math.complex
```

An import may introduce a source-local qualifier without changing the logical module or dependency identity:

```
import std.math.complex as cx

in sample : cx.Complex<u8>
out total : cx.Complex<u9>
total = cx.complex_sum([sample, sample])
```

The qualifier applies to types, functions, and struct constructors declared by that exact logical module (`cx.Complex { re = ... im = ... }` is valid). It is erased before semantic typing, so the qualified and historical unqualified spellings have the same canonical identity. An aliased import does not expose those declaration names unqualified and does not re-export declarations from a transitive dependency. Aliases are not runtime values or filesystem names; wildcards, member renaming, and re-export remain unsupported.

The compiler-shipped `std` namespace keeps its existing resolver. Other package names are resolved only when the source belongs to a locked project.

4.1 Project manifest

A project is rooted by a versioned `zlang.toml`:

```
schema = 1

[project]
name = "demo"
version = "0.1.0"
source-root = "src"

[dependencies]
acme = { path = "../acme-dsp" }

[dependencies.bus_models]
git = "https://example.invalid/hardware/bus-models.git"
rev = "0123456789abcdef0123456789abcdef01234567"
```

The `schema` field is required project metadata: it tells `zlang lock update` how to interpret the manifest. Use the value shown for the current project format.

Imports use logical package and module names rather than absolute checkout or cache paths. A file `filters.zhl` directly below package `acme`'s source root is imported as `acme.filters`; nested directories append dotted components.

Path locators are relative to the manifest containing them. Git revisions must be full lowercase 40- or 64-digit hexadecimal object IDs. Branch names, tags, and abbreviated revisions are deliberately rejected because they are mutable.

4.2 Lock update

Dependency resolution and fetching are explicit:

```
zlang lock update --project zlang.toml
```

The command validates the complete transitive graph before publishing a deterministic `zlang.lock`. It records each package manifest, exact source-module index, imports, content digests, and dependency edges. Git content is populated in the project cache during this command. Path dependencies remain at their manifest-relative location and are checked byte-for-byte against the lock.

Resolution rejects:

- dependency cycles and conflicting package/module declarations;
- source-root traversal or symlink escape;
- a package whose declared identity does not match its dependency key;
- missing, added, removed, or modified locked `.zhl` modules;
- modified dependency-resolution fields in dependency manifests (profile-only edits are intentionally outside the resolution identity);
- missing Git cache content or a revision mismatch.

The lock is written only after the entire graph validates. Failed updates leave the previously accepted lock usable.

Scenario	What to do	Without it
Standalone source outside any project, including compiler-shipped `std.*` imports	Compile directly; no project lock is needed.	No consequence.
Project with zero external dependencies	Create and retain `zlang.lock` for its manifest.	A source belonging to a project still requires that project's valid lock.
Project with local path packages	Run `zlang lock update` after declaring dependencies and whenever their source or resolution fields change. Keep `zlang.lock` with the project.	Compilation rejects a missing or stale lock; it never silently uses changed dependency files.
Project with pinned Git packages	Run `zlang lock update` while Git/network access is available; retain the exact revision and local cache for later offline builds.	Ordinary compilation does not fetch; missing cache or changed revision fails.
CI or another machine	Supply the project manifest, its lock, exact path sources and cached Git revisions (or update the lock before the offline build).	A lock file alone does not provide Git source bytes; compilation fails closed.

`zlang lock update` resolves dependencies; it is not a package installer or a step required for every single-file invocation.

Bounded scalar `extern module` implementations may also be declared under `[external-mappings.NAME]` and selected by a profile's `external-mappings = ["NAME"]`. The lock records every HDL source path and SHA-256. Paths are project-relative, must remain inside the project, and may not be symlinks. The mapping changes generated SystemVerilog without changing the behavior expressed by the ZLang source.

4.3 Optional exact compiler and stdlib lock

Project schema 1 and lock schema 2 remain supported. A project may instead use schema 2 to pin the compiler and bundled standard library as well as its source dependencies. Inspect the SDK you actually intend to use:

```
zlang lock toolchain
zlang lock toolchain --json
```

The first command prints the three requirement fields. Copy their exact values into a `[toolchain]` table: `compiler-version`, `compiler-identity`, and `stdlib-identity`; then run `zlang lock update`. This produces lock schema 3 with the checked compiler/runtime and stdlib inventories. Both the version and identities must match. A pinned dependency also requires a schema-2 root with compatible requirements. The reserved `std` namespace remains bundled with the SDK, rather than becoming a separately fetched package.

Missing or changed SDK inputs reject compilation and cached tooling reuse. Restart the compiler/LSP after replacing compiler files. Selecting a different SDK is an explicit requirement update and requalification, followed by lock regeneration; the lock command does not install or upgrade a compiler. Earlier a24 binaries may predate this opt-in contract: use a matching build of the current source SDK, not merely another binary with the same version label.

4.4 Offline compilation

Ordinary compilation never fetches dependencies and never updates project metadata:

```
zlang src/top.zhl --project zlang.toml --check
zlang src/top.zhl --project zlang.toml --systemverilog build/top.sv
```

`--project` may name a manifest or its directory. Without it, `zlang` searches the source file's parent directories for `zlang.toml`. If no project is found, single-file and compiler-shipped `std.*` compilation retain their existing behavior; arbitrary external imports remain an explicit error.

Compilation checks the current manifest, lock, every path dependency, and every cached Git dependency before checking the source. It is read-only: unavailable or dirty content is a lock mismatch, not an invitation to fetch or rewrite files. Build outputs and ZLang output/cache directories must remain outside the resolved root, manifest, lock, dependency, and stdlib inputs.

4.5 Identity and artifacts

The exact locked dependency closure is carried through semantic and canonical IR and BackendArtifact JSON. It contributes to selected/build identities, formal proof-cache keys, and synthesis-cache keys. Changing dependency content therefore invalidates results even when emitted RTL text happens to remain identical.

`artifact_hash` keeps its narrower meaning: the hash of emitted backend text. The separate build identity combines that text with the logical root module and locked semantic closure. This distinction permits byte-identical RTL to be recognized while preventing evidence from one dependency closure being reused for another.

4.6 Deliberate boundaries

The current project model has no registry or semantic-version solver, editable global packages, implicit network access, Git submodules/LFS/subdirectories, wildcard imports, member renaming, or re-export. Implementation profiles are a separate set of build settings and do not change dependency resolution. Path dependencies declared by a Git package are also deferred in this bounded slice; use another pinned Git dependency instead of reaching outside a fetched checkout.

Ordinary compilation remains offline; resolved module contents and their recorded digests define the accepted dependency contents.

5 Native simulation

Native simulation runs a checked design without first generating SystemVerilog or starting an external HDL simulator. Use the Python API when a test needs direct input, state, and clock control:

```
from zlang import sim

program = sim.compile("examples/counter.zhl", top="Counter")
instance = program.create()
instance.tick("clk")
print(instance.get("y"))
instance.close()
```

For ordinary command-line use, choose either a bounded single-clock run or an explicit event schedule:

```
zlang sim examples/counter.zhl --top Counter \
  --clock clk --cycles 100 --json
zlang sim examples/counter.zhl --top Counter \
  --events examples/counter.events.jsonl --trace build/trace.fst
zlang sim examples/add.zhl --top Add --set a=200 --set b=55 \
  --compare-with iverilog --compare-artifacts build/sim-compare --json
```

Use `--set NAME=JSON` for ordinary typed binary inputs. Repeat it for multiple ports. `--cycles N --clock NAME` is a convenience for one-clock designs whose reset/input state is already suitable. Use `--events` for resettable designs, multiple clocks, coincident edges, or changing inputs. For `examples/counter.zhl` (`Counter`), this schedule resets, releases, then advances four more edges:

```
{"reset":{"rst":true},"edges":["clk"]}
{"reset":{"rst":false},"edges":["clk"]}
{"edges":["clk"],"repeat":4}
```

`set` changes input values before the event. `reset` uses assertion booleans: `true` asserts the named reset and `false` releases it, independently of the declared physical polarity. For an active-low reset, `true` therefore drives the pin low. `edges` contains every clock edge committed atomically from one pre-edge snapshot; therefore coincident clocks belong in the same array. Synchronous reset takes effect on its domain edge, while supported asynchronous assertion takes effect in the event that changes the reset level.

Each non-empty `--events` JSONL line is one simulation event. An event may add `"repeat": N` to apply the same updates, resets and edges for `N` consecutive events; `N` must be a positive integer and the expanded schedule is bounded to 1,000,000 events. The CLI retains one parsed mapping and repeat count per JSONL record rather than copying its nested payload `N` times. This keeps long idle or backpressure intervals declarative without requiring a project-specific event generator.

5.1 Four-state initialization and diagnostics

Ordinary mode uses deterministic binary-zero startup for state without an initializer. With `--logic-state`, register bits without an initializer start at `U`, meaning they have never been refreshed; declared initial values remain known. A write derived from unresolved state produces `X`; exact controlling values still resolve (`U | 1 == 1`, `X & 0 == 0`). These are simulation diagnostics, not electrical `Z`, drive strength, or the Verilog event-region model.

Packed four-state spellings are MSB-first strings containing `0`, `1`, `u`, or `x`. Use `--set-logic NAME=01ux` for an input and `--initial-reg NAME=01ux` to override one named register path before the first event. The width must match exactly. `--strict-uninitialized` implies logic-state execution and fails when a typed observation requires an unresolved `U` or `X`; it does not alter compilation or Direct-SV.

Both modes execute the selected implementation. U/X tags describe diagnostic uncertainty, not additional physical hardware values. Legal exact binary optimizations can change U/X propagation, so the waveform is not a record of every operand in the original source spelling. Strict mode checks unresolved typed observations and stateful controls; resolved cancellation or controlling values need not fail. Traces and `get_logic()` retain tags for inspection.

5.2 FST/VCD waveforms and differential RTL comparison

`--trace PATH.fst` writes the preferred compact native waveform; `--trace PATH.vcd` retains VCD interoperability with external and formal tools. The suffix selects the format. Other suffixes, including `.ghw`, are rejected. Physical clocks and resets are always present; repeat `--trace-signal NAME` to keep only selected data/state signals. The timeline uses two event-domain ticks per simulation event and reflects the declared edge/reset contract. It is an event visualization, not an analog delay model or physical frequency measurement. A serialized file timescale does not turn native events into hardware nanoseconds. Standard FST and VCD both collapse unresolved provenance to `x`, so ZLang emits hidden `__zlang_meta/*_u_mask` companions that its query service combines back into distinct `U` and `X` values.

5.2.1 Compare native and RTL from one command

The native engine and both RTL comparisons share the `zlang sim` entry point. ZLang generates Direct-SV, an event-driven testbench and physical input stimulus, invokes the requested simulator, and compares output samples automatically. Icarus requires `iverilog` and `vvp` on `PATH`. Verilator requires `verilator` and its C++ build tools. Use the matching native runtime from the toolchain installation.

Run the following from the checkout root after activating its `.venv`. Each backend gets its own fresh artifact directory.

```
run_dir=$(mktemp -d /tmp/zlang-XXXXXX)

# Native execution
zlang sim examples/counter.zhl --top Counter \
  --events examples/counter.events.jsonl --json

# Native versus Icarus Verilog
zlang sim examples/counter.zhl --top Counter \
  --events examples/counter.events.jsonl \
  --compare-with iverilog --compare-artifacts "$run_dir/iverilog" --json

# Native versus Verilator
zlang sim examples/counter.zhl --top Counter \
  --events examples/counter.events.jsonl \
  --compare-with verilator --compare-artifacts "$run_dir/verilator" --json

printf '%s\n' "$run_dir"
```

The checked-in schedule contains 21 expanded events, including reset, and every run finishes with `{"y":20}`. With `--json`, stdout contains the output snapshot for each event. A zero comparison exit code confirms that both executions completed with matching samples.

Each comparison directory retains `design.sv`, `compare_tb.sv`, `zlang-compare-events.hex`, `compile.log`, `run.log` and, on success, `comparison.json`. The JSON record includes product identities, the event count and native/RTL samples. `--compare-timeout` bounds each external compile/run command, with a default of 120 seconds. For a native waveform, add `--trace "$run_dir/native.fst"` to the native command.

Ordinary comparisons use binary simulation. Four-state diagnostic options have their separate native contract above. Coverage follows the supplied event schedule. Formal verification has its own proof workflows. For typed file input and transaction logs, use the simulation I/O example.

Comparison checks every physical output after every event. A mismatch, X/Z value, missing sample, tool failure, or timeout fails closed. Comparison artifacts are retained only when an

empty `--compare-artifacts` directory is supplied. Sequential RTL must receive the same explicit reset events needed by physical hardware; use `--events` when comparing a resettable design.

Exit status is nonzero for compilation/plan errors, native runtime failures, strict-U/X violations, verification/protocol failures, comparison mismatch, external tool failure, or timeout. Machine consumers should use `--json` and still treat the process status as authoritative. `--compare-artifacts` must name a new empty directory; without it, temporary comparison material is discarded.

One instance is not thread-safe; independent instances may run concurrently. `eval()` settles combinational logic without advancing a clock. Reset assertion follows the declared synchronous/asynchronous contract; it is not an ordinary combinational input update. `edge_many()` computes all next-state values from one pre-edge snapshot and commits atomically. Synchronous resets act on their domain edge, while supported asynchronous assertion acts immediately. Values are exact-width and are never silently truncated.

The following are implementation limits of this compiler release's native simulator, not ZLang language semantics. Native simulation accepts at most:

- 32,768 compiled simulation operations;
- a cumulative packed-value work budget of 32,768 machine words;
- 16 MiB of compiled simulation data;
- packed values up to 8,192 bits;
- arithmetic operands up to 512 bits; and
- memory elements up to 8,192 bits.

Generated functional work is also bounded to 8 nested regions, 1,000,000 total iterations, and an estimated runtime-work budget of 8,000,000 operations. Compilation fails with a diagnostic when a limit is exceeded.

There is no silent fallback to an RTL simulator. Native simulation requires the matching Linux x86-64/WSL2 wheel described in the installation chapter; an unsupported design or missing runtime produces an error.

5.3 Debug without an event file

The Linux-x64 VS Code extension can run a bounded `clock/cycles` launch without JSONL and edit ordinary input ports while paused. It does not generate a reset sequence; use JSONL for reproducible reset/input schedules or multiple clocks. The manual-input debugger workflow owns the launch configuration, F9/Continue/source highlight and live-waveform steps. `events` and `clock/cycles` remain mutually exclusive.

5.4 Python simulation without an event file

The Python API can drive inputs, reset and clock edges directly, without an event file. See the complete Python testbench and its expected counts. Python breakpoints run in a separate simulation instance, not the ZLang source debugger or its live waveform. Export JSONL from the testbench when the same stimulus must be replayed there.

5.5 Simulation-only typed file I/O

Use simulation-only source I/O for conditional typed JSONL reads, pre-edge structured logs and explicit EOF behavior. Activation requires a named `simulation` block and channel bindings; inactive blocks do not alter hardware. The runnable `examples/simulation_io.zhl` includes reset, stalls and drain; VS Code provides **Debug simulation I/O**. Use fresh file outputs or console `-`.

6 Debugging native simulation in VS Code

> a24 status: compiler debug-bundle support and the Linux-x64 native adapter / waveform implementation are a bounded preview. The universal VSIX remains editor/LSP-only. The enhanced Linux-x64 VSIX contains two reviewed native executables and no Rust implementation source.

This guide describes the user workflow. The debugger runs the same native simulation plan as `zlang sim`; it does not synthesize test vectors or infer a clock/reset sequence. Stimulus comes from either:

- a user-authored JSONL event file, normally kept in a project's `simulation/` directory;
- a JSONL file produced by the project's existing test-vector generator; or
- a bounded `clock` plus `cycles` launch, with ordinary input ports editable while paused. This mode does not generate a reset sequence.

The same JSONL file can be used by the command-line simulator, the VS Code debugger, and native-versus-RTL comparison. Keeping stimulus outside the debugger makes a failing run reproducible without VS Code.

6.1 Prerequisites

Use the Linux-x64 VSIX; the universal VSIX contains editor and LSP support but not the native debug adapter. The compiler, native runtime, language server, VSIX, and bundled debug tools must come from the same ZLang HDL release.

From a source checkout:

```
make install
source .venv/bin/activate
.venv/bin/zlang-lsp --version
```

Install the matching `zlang-hdl-0.1.0-linux-x64.vsix`, set `zlang.lsp.path` to `${workspaceFolder}/.venv/bin/zlang-lsp`, and reload the VS Code window. Save every relevant `.zhl` file before starting a session. The compiler rejects a debug launch when an open source buffer and the saved source snapshot differ.

6.2 Quick start

The clone includes `.vscode/launch.json` and `examples/rule_action.events.jsonl` for `examples/rule_action.zhl`. Open the repository folder in VS Code; there is no event file to generate before pressing F5. The checked-in schedule is:

```
{"reset":{"rst":true},"set":{"enable":0},"edges":["clk"]}
{"reset":{"rst":false},"edges":["clk"]}
{"set":{"enable":1},"edges":["clk"],"repeat":2}
{"set":{"enable":0},"edges":["clk"]}
```

The checked-in launch configuration is shown below. In another workspace, add an equivalent entry to its existing `.vscode/launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "zlang-hdl",
      "request": "launch",
      "name": "Debug RuleAction",
      "program": "${workspaceFolder}/examples/rule_action.zhl",
      "top": "RuleAction",
      "events": "${workspaceFolder}/examples/rule_action.events.jsonl",
      "logicState": true,
      "strictUninitialized": false,
      "initialRegisters": {},
      "traceSignals": ["clk", "rst", "enable", "fired", "count"],
      "stopOnEntry": true
    }
  ]
}
```

Open `examples/rule_action.zhl`, **Save All**, select **Run and Debug → Debug RuleAction**, and press **F5**. The session pauses before the first event. **F10 / Step Over** advances one complete simulation event, not one source statement. The five steps produce `count = 0, 0, 1, 2, 2`: reset, release, two enabled increments, then hold. The declared `= 0` also sets the initial `count` value to zero; uninitialized state without a reset value is `U`. Click the gutter to the left of line 10 (`when enable`), or place the caret on that line (or the `count <- ...` line inside it) and press **F9**, to toggle a source breakpoint. Press **F5 / Continue** after the entry stop: the debugger stops after event three with `count = 1`; Continue again stops after event four with `count = 2`. The breakpoint is solid red once verified; a hollow marker means that line has no executable site. On a hit, VS Code highlights the selected source line and shows its execution arrow (the exact colors follow your theme). The highlight marks the completed atomic event, not a partially executed statement. It stays while you inspect values and clears when execution resumes. Several breakpoints in one action stop once at the first selected line; they do not split the hardware event. No `debug.allowBreakpointsEverywhere` setting is needed. **Restart** starts over; **Shift+F5** ends the session.

Use **ZLang HDL: Open Live Waveform** from the Command Palette to inspect the active session beside the source. Keep **Follow Live** enabled so cursor A and its value column follow each step. Signals named in `traceSignals` have history from event zero. While paused, right-click a value in **Variables** or **Watch** and choose **Add to Waveform**. A newly added signal records from the next event; restart the session when complete history is required. A saved FST/VCD tab is separate from this live view and does not follow debugger steps.

Clocks and resets are recorded automatically from event zero, even when omitted from `traceSignals`. For RuleAction, the live viewer includes `clk` and `rst`; they are also available under **Variables → Clocks and resets** and in **ZLang HDL: Add Live Trace Signals**. A saved layout still controls which rows are displayed. If it predates these controls, or a row was removed, click **+** beside `clk` or `rst` in the live Design Browser to show it again.

Clock pulses follow the declared rising or falling edge. Reset levels follow the declared active-high or active-low polarity; JSONL `true` means *asserted*, not necessarily a physical `1`. Within event `n`, reset changes appear at tick `2n−1`, the clock edge and resulting values at `2n`, and the clock returns to idle at `2n+1`. Coincident clocks share the same edge tick. This is an event visualization, not a physical delay or frequency model. The clock/reset scope is read-only: change reset through the event file, not by editing its displayed value. After updating compiler and VSIX together, reload the VS Code window and start a new debug session.

6.3 JSONL event files

Each non-empty line is one atomic event object:

- `set` assigns ordinary typed input values;
- `set_logic` assigns packed `0`/`1`/`U`/`X` values;
- `reset` asserts or releases named physical reset inputs;

- `edges` lists clock edges that occur atomically from one pre-edge snapshot;
- `repeat` repeats that event without materializing duplicate event objects.

Inputs and reset levels remain unchanged until a later event replaces them. `repeat` must be positive, and one run is bounded to one million expanded events. Errors retain the JSONL source line so a malformed or invalid vector can be fixed at its origin.

Before opening VS Code, validate the exact stimulus from the terminal:

```
.venv/bin/zlang sim examples/rule_action.zhl --top RuleAction \
  --events examples/rule_action.events.jsonl --logic-state --json
```

For generated vectors, make the generator write the same JSONL schema and check the generated file into the project only when it is stable test data. Transient or large generated schedules belong under ignored `build/` output. The debugger never runs an arbitrary vector-generator command as part of launch.

6.4 Debug without an event file

For an interactive single-clock experiment, add this entry to the `configurations` array in `.vscode/launch.json`. It needs no JSONL file:

```
{
  "type": "zlang-hdl",
  "request": "launch",
  "name": "RuleAction - manual inputs",
  "program": "${workspaceFolder}/examples/rule_action.zhl",
  "top": "RuleAction",
  "clock": "clk",
  "cycles": 20,
  "logicState": true,
  "stopOnEntry": true,
  "traceSignals": ["enable", "fired", "count"]
}
```

1. Save `examples/rule_action.zhl`, select **RuleAction - manual inputs**, and press **F5**. The debugger stops before the first clock event.
2. Expand **Variables > Ports**, right-click `enable`, choose **Set Value**, and enter `1`.
3. Press **F10** twice. **Registers > count** shows `1`, then `2`.
4. Change `enable` to `0` and step again. `count` holds at `2`.
5. Set `enable` back to `1`, place an **F9** breakpoint on `when enable` or its `count <- ...` action, then press **F5 / Continue**. The next enabled event stops with `count = 3` and highlights the selected source line.
6. Open **ZLang HDL: Open Live Waveform** and keep **Follow Live** on to observe the same events. Editing an input does not advance the clock; its next recorded sample appears when another event executes.

`count` starts at its declared `= 0`; this example does not assert reset. Clocks, resets, registers and memories are read-only in Variables. Use JSONL for reset assertion/release, coincident clocks, automated input changes or a reproducible protocol exchange. `events` and `clock`/`cycles` are mutually exclusive. The session ends after 20 clock events; increase `cycles` and relaunch for a longer experiment. Restart restores the declared initial state, not a recording of manual input edits.

6.5 Python simulation without an event file

For programmatic stimulus, a Python testbench can drive the native simulator directly. Save this as `tb_rule_action.py` in the repository root:

```python
from zlang import sim

with sim.load("examples/rule_action.zhl", top="RuleAction", logic_state=True) as dut:
    dut.set("enable", 0)
    dut.reset("rst", asserted=True)
    dut.edge("clk")
    assert dut.get("count") == 0

    dut.reset("rst", asserted=False)
    dut.edge("clk")
    for enable, expected in [(1, 1), (1, 2), (0, 2)]:
        dut.set("enable", enable)
        dut.edge("clk")
        count = dut.get("count")
        print(f"enable={enable}, count={count}")
        assert count == expected
```

Run it from the repository root with `.venv/bin/python tb_rule_action.py`. The final three counts are `1`, `2`, `2`. In VS Code, use the Python Debugger with this checkout's `.venv/bin/python` to put breakpoints in the Python file. F10 there steps Python statements; `dut.edge("clk")` advances one hardware event.

This Python API creates a separate simulation instance. It does not connect to a running ZLang debug session: `.zhl` breakpoints and **Open Live Waveform** do not follow it. For those features, use a `zlang-hdl` launch with manual inputs as above, or replay a Python-generated JSONL schedule. The ZLang debugger does not execute the Python testbench itself.

6.6 Breakpoints, stepping, and values

- Source breakpoints are valid only on compiler-published `when` rules, FSM transitions, and verification clauses. VS Code leaves other breakpoints unverified instead of pretending they can execute.
- One DAP thread represents the simulation. Stack frames identify the active hierarchy/source event; they are not software call frames.
- **Continue** runs until a verified source/data breakpoint, exception, or the end of the bounded event schedule. **Pause** takes effect after the current atomic event.
- Data breakpoints stop when the selected signal, register, or memory cell changes. Registers and memories are read-only. Input ports may be changed while paused.
- Assertion, protocol, strict-U/X, and runtime failures are debugger exceptions with compiler-owned source attribution.
- `initialRegisters` accepts the same resolved register names and values as the simulation initial-state interface. Use it to reproduce a deliberate initial condition, not to hide a missing reset.

6.7 Live and saved waveforms

While paused, expand a port, register, or memory in **Variables**. Right-click the value or one of its fields and choose **Add to Waveform**. The same command is available on a signal in **Watch**. Watch accepts exact published signal paths, such as `packet.mode` or `buffer[3].value`, not arbitrary expressions. The live panel opens automatically. **ZLang HDL: Add Live Trace Signals** offers a multi-select list of whole ports/registers/interfaces.

- Adding a struct, tuple or vector adds its packed row and typed child rows. Select a field/index instead when only that part is needed. Large arrays are paged in Variables; whole selections exceeding 4096 trace rows are rejected.
- Enum rows show their declared member and exact code, including sparse codes. Invalid codes are labelled invalid; U/X values are not decoded as members.
- Signed integers retain signedness. Fixed-point rows show the exact scaled value as `raw/2^fraction` together with its raw bits. Select Hex/Binary to inspect the packed representation. Struct/tuple/vector fields use the same compiler-owned layout as execution, with no host-word-width truncation.

- Tagged unions show the tag and per-variant fields. Inactive fields say **inactive variant**; an unknown tag says **unknown variant**. These labels are presentation states, not extra logic values or guessed payloads.
- Interfaces are groups, not packed structs. Their payload/control fields keep their individual directions and clock domains. Ready/valid includes payload, valid and ready; aggregate bus interfaces retain their channel groups.
- Expand a memory and select individual cells/fields. The entire memory is not implicitly recorded. Registers and memory remain read-only in the debugger.

Type is the default format for these added live rows. Adding a signal during a run starts recording at the next atomic event. Earlier history is **not recorded**, rather than U or X; restart for complete history. A whole selection is validated before its trace configuration changes. Multiple field rows share one recorded backing value.

Typed metadata is a debugger overlay, independent of the production simulation plan and RTL. The compiler and native adapter must support the same overlay version; update the matching VSIX when updating the compiler. Saved VCD/FST exports currently contain recorded raw backing signals. They do not preserve the live catalog's nominal struct/union types; external enum metadata is used only when actually present in the file. Session sidecars save presentation, not type guesses or waveform samples.

The waveform workspace guide covers hierarchy browsing, groups, multiple viewports, measurements, navigation, and portable sessions.

Live and saved views share these controls, including the 16-color palette, row height, duplicate rows, Cut/Copy/Paste, search and measurements. In the live toolbar, **Add signals** opens the trace selector and **Refresh** only refreshes recorded data; it never advances or restarts simulation. **Save Session** saves a layout template for another live session; **Load Session** restores it using exact recorded signal names and widths. The workspace also recovers the layout when the panel is reopened. **Export Trace** saves FST/VCD data for inspection after the debug session ends. Late-added history is still not backfilled.

The live panel follows the current debug session. The readonly waveform editor opens saved `.fst` and `.vcd` files independently of a debug session. FST is preferred for compact native traces; VCD remains useful for interchange with external RTL and formal tools.

- Click a row to select and highlight a signal.
- **Change**, its direction buttons, and Left/Right move among changes of the selected signal only.
- Mouse-wheel zooms around the pointer; **Shift+wheel** pans in time.
- Hold the middle mouse button and drag left/right to pan in time or up/down to scroll signals.
- Click places cursor A; **Shift+click** places marker B.
- Double-click restores the complete trace.

Native waveform time is event-index visualization: two ticks represent one simulation event. Native traces are labelled `zlang_event_ticks` and have no physical frequency measurement. External VCD/FST retains its declared timescale.

6.8 Project launches

For a project root, add `project` and optionally `profile`:

```
{
  "type": "zlang-hdl",
  "request": "launch",
  "name": "Debug project top",
  "program": "${workspaceFolder}/src/top.zhl",
  "top": "Top",
  "project": "${workspaceFolder}/zlang.toml",
  "profile": "portable",
  "events": "${workspaceFolder}/simulation/events.jsonl",
  "traceSignals": ["input.valid", "input.ready", "output.valid"]
}
```

Project compilation validates the existing `zlang.lock` read-only. Debugging does not fetch dependencies or update the lock.

6.9 Troubleshooting

- **No debug configuration:** confirm that the Linux-x64 VSIX is installed, not only the universal editor/LSP package.
- **Language server unavailable:** run `.venv/bin/zlang-lsp --version` and inspect **View → Output → ZLang HDL** for the exact executable path.
- **Save All before debugging:** save the relevant `.zhl` files and relaunch; the adapter will not combine a stale compiled bundle with dirty source.
- **Breakpoint is unverified:** move it to a rule, FSM transition, or verification clause published by the compiler.
- **No waveform history:** list the signal in `traceSignals`, or add it live and restart when history from event zero is needed.
- **Runtime/adapter version mismatch:** install compiler, native wheel, and Linux-x64 VSIX from the same release.

6.10 Architecture and contracts

ZLang HDL keeps one language implementation. The Python compiler and `zlang-lsp` parse, type-check, select implementations, and publish an immutable debug bundle. Rust executes the ordinary `SimulationPlan`, stores four-state values, serves the Debug Adapter Protocol (DAP), and indexes waveforms. The adapter does not parse ZLang source or reconstruct compiler semantics.

```
saved .zhl + project snapshot
    -> zlang-lsp: zlang/prepareDebugBundle
    -> ordinary SimulationPlan + debug overlay
    -> zlang-debug-adapter (DAP over stdio)
    -> simulation-core + waveform-core
```

The contracts are versioned independently:

- `zlang-native-debug-bundle-v1` describes the compiler product;
- `zlang-debug-overlay-v1` identifies executable source sites separately from the production plan;
- `zlang-debug-adapter-v1` identifies the DAP executable contract;
- `zlang-waveform-trace-v1` identifies live range-query results.
- `zlang-waveform-query-v3` identifies bounded readonly FST/VCD hierarchy, value, search and digital-measurement queries.

Presentation paths do not participate in stable compiler identities. Debug sites and probes do not alter semantic identity, selected implementation identity, or the ordinary simulation-plan identity. Relevant `.zhl` buffers must be saved before launch so source digests and breakpoints refer to the same snapshot.

6.11 Initial supported surface

The Linux-x64 VSIX contributes debug type `zlang-hdl`. One thread represents one simulation. A step completes one atomic evaluation, reset, or clock event; there is no fictional software call stack. Source breakpoints are accepted only at compiler-published rule, FSM-transition, or verification sites. Data breakpoints observe signal changes. Registers and memories are read-only; input ports may be changed while paused. Runtime, assertion, protocol, and strict-U/X failures stop as DAP exceptions.

An event file is mutually exclusive with a bounded `clock` plus `cycles` launch. Reverse execution, attach, unbounded run, arbitrary state mutation, and expression-level stepping are not supported.

Restart reuses the frozen bundle, resets execution and waveform history, and reapplies the declared initial-register overrides and trace selection.

6.12 Waveforms

The live panel reads bounded delta ranges from the active debug session. The readonly custom editor opens both `.fst` and `.vcd` through one native query service. FST is preferred for new native traces; VCD remains a permanent interchange and formal-tool path. Signals are loaded only when displayed or searched. The viewer provides hierarchy/filtering, drag ordering, nested named groups, one to four independent viewports, cursors, transition/value search, binary/hex/signed display, and pointer-centered zoom/pan. Layout persists per waveform URI. Standard FST/VCD collapse unresolved provenance to `x`; ZLang's hidden U-mask companion is combined by the query service so the viewer renders `U` and `X` distinctly.

Optional Waveform Control Protocol v0 support is disabled by default. In a trusted workspace, set `zlang.waveform.wcp.enabled` to start a bounded listener on `127.0.0.1`; `zlang.waveform.wcp.port` selects a fixed port or `0` requests an ephemeral one. The output channel reports the chosen address. The listener does not bind non-loopback interfaces and accepts only the documented v0 greeting and command set.

Time is event-index visualization: two event-domain ticks per simulation event. It is not an analog timing model or physical frequency measurement. Trace storage is delta encoded, chunked, capped at one million events, and may spill beneath VS Code `globalStorageUri`.

6.13 Packaging boundary

The universal VSIX remains editor/LSP-only. The enhanced Linux-x64 VSIX adds the stripped `zlang-debug-adapter` and `zlang-waveform-indexer` ELF executables. It ships no Rust/TypeScript source, Cargo metadata, debug symbols, source maps, runtime downloader, or private build paths. The two binaries are admitted by exact path, size and SHA-256 in `release/status.json`; public CI audits those bytes before placing them in the VSIX.

Install the platform package downloaded from the matching GitHub release:

```
code --install-extension zlang-hdl-0.1.0-linux-x64.vsix --force
```

The public source checkout can reproduce the package envelope from the same reviewed executable bytes with `make editor-native-host-test`. Rebuilding the native Rust implementation is deliberately a release-host operation and is not presented as a public-source command.

6.14 Source simulation channels

Use simulation-only source I/O for conditional typed JSONL reads, pre-edge structured logs and explicit EOF behavior. Activation requires a named `simulation` block and channel bindings; inactive blocks do not alter hardware. The runnable `examples/simulation_io.zhl` includes reset, stalls and drain; VS Code provides **Debug simulation I/O**. Use fresh file outputs or console `-`.

7 Waveform workspace

The Linux-x64 extension displays saved VCD/FST files and native debugger traces with the same waveform controls. Install matching compiler, native runtime, and extension binaries. The universal VSIX provides editor/LSP support only.

A saved `.fst` or `.vcd` tab displays a file snapshot. It does not attach to the debugger or follow simulation steps. Use **ZLang HDL: Open Live Waveform** to observe the active ZLang debug session; use **Reload** to reread a changed file.

7.1 Refresh a simulation result

When a simulator writes or replaces an open FST/VCD file, **File changed** appears beside the highlighted **Reload** button. Wait for simulation to finish, then click **Reload**. Signals, duplicate rows, colors, groups, and viewports are kept; cursors and ranges are clamped if the new trace is shorter. Reload updates all editors of the same trace. It does not reread a partially written file automatically; native queries pause until Reload so old and newly written data are not mixed. If reading fails, the previous view remains and Reload can retry.

Signals absent from the new catalog remain as removable **holder** rows. Their previously loaded diagrams are faded and explicitly marked `holder`; they are not live/current samples and cannot drive navigation or measurements. **×** deletes a holder. Exact name-and-width reappearance rebinds the original row on the next reload.

Only the last loaded visible tiles are retained, with a 16 MiB accounting cap per editor. Outside those ranges, or after eviction/reopening a session, the holder reports that its snapshot is unavailable. Sessions still store no waveform samples. In a live debugger trace the button is **Refresh**: it rereads recorded history without executing an event, resetting state, or replaying inputs.

Live debugger traces include the design's clocks and resets automatically. Their waveforms use the declared clock edge and reset polarity. They can also be added from **Variables → Clocks and resets → Add to Waveform** or the live signal picker. Native trace ticks describe atomic events, not hardware time: reset changes precede the event edge by one tick, and clock pulses return to idle one tick after it. Reset-only events do not create clock pulses.

7.2 Open and arrange signals

Run **ZLang HDL: Open Waveform** or open an `.fst`/`.vcd` file. If another viewer owns the file, choose **Reopen Editor With → ZLang HDL Waveform**. The simulation guide explains how to generate traces.

The Design Browser shows the trace hierarchy. Expand scopes or filter by name. Large signal lists load in pages of 200. Click **+** beside a signal or double-click its name to add a row. Each addition creates another independent row, even when that signal is already displayed. A scope's **+** adds its direct signals; **Deep** includes descendants. A selection exceeding 4096 signal rows (including copies) is rejected, so choose a smaller scope. Ctrl/Cmd-click or Shift-click browser rows to select several and drag them into the signal list or a group.

Click a displayed signal to make it active. Its name and waveform are highlighted; the footer identifies it. Ctrl/Cmd-click or Shift-click selects several for presentation changes. The last selected signal controls navigation and measurements.

Drag pane boundaries to resize the browser, signal list, waveform, and Inspector. **Inspector** collapses or reopens measurements; on narrow windows it opens over the right side. **Group** and **Divider** add named sections. Drag rows to reorder or nest groups. Click a group arrow to collapse it, double-click its name to rename it, or use **×** to remove a row.

The Inspector offers a 16-color palette and a **Default** color reset. **Row height** is a numeric field with up/down arrows, from 24 to 128 pixels. Color and height apply to the selected rows; **Format** selects binary, hex, unsigned, signed, or enum display for those rows. U/X/Z retain their distinct state colors. Right-click the signal list or a waveform row for **Cut / Copy / Paste**, or use **Ctrl/Cmd+X / C / V**. These operations copy selected rows with their appearance settings, groups, and dividers through the system clipboard. Paste inserts after the active row, inside an active group, or at the end when nothing is selected. Cut removes rows only after a successful clipboard

write. Shortcuts in text fields continue to edit text normally. **Shift+F10** opens the row menu. The VS Code Command Palette also offers **ZLang HDL: Cut Waveform Rows**, **Copy Waveform Rows**, and **Paste Waveform Rows** while a waveform is active. Copies have independent color, height, alias, and format, but share the underlying trace data and queries. Removing one copy does not remove the others. Pasting into another trace rebinds exact hierarchical names and widths; absent signals become holders. Clipboard data contains no waveform samples. A holder's saved diagram is retained only within the editor that already has its tiles. Clipboard transfers are bounded to 1 MiB and the usual row/group limits. **Alias** and **Mnemonics** edit the active row. Enum display uses trace-provided labels when available. A custom mnemonic map uses exact bit strings, for example `{"00":"Idle","01":"Header","10":"Payload"}`.

Split+ creates up to four time viewports. Click one to make it active. With **Sync** enabled, panning or zooming applies the same interval to all viewports. Otherwise each retains its own range. **Split−** removes the active viewport.

7.3 Numeric analog display

Display: Auto / Digital / Analog controls waveform shape independently of **Format**, which controls value text. Auto plots multi-bit typed integers, `fixed`/`ufixed` fields and file `real` channels. Clocks, one-bit handshake controls, raw bit buses and enums retain digital shapes. Aggregate fields use their own leaf types; neither signal names nor bit patterns determine a type.

For example, the live CORDIC `fixed<16,14>` sine/cosine fields are signed and scaled by `2^-14` automatically: raw `0x4000` plots as `1`, `0xC000` as `-1`. The input phase remains a signed integer; the viewer does not guess physical units. Auto uses the compiler catalog or explicit compatible file type hints. An ordinary saved VCD/FST bus often has no signedness/fraction metadata; choose Analog and set **Interpretation** and **Fraction bits** in the Inspector in that case. A blank fraction override uses the declared scale, or zero for raw buses. These per-row settings survive session files, autosave and Cut/Copy/Paste.

Analog lanes auto-scale the fetched visible values and use sample-and-hold steps. They do not insert samples, smooth data, gate ready/valid payloads or infer transfers. U/X/Z, inactive variants and non-finite/unrepresentable values break the numeric trace. Plot coordinates/axis labels use floating-point presentation; the original samples and raw/exact typed formats remain available. Analog rows default to 72 pixels; the existing row-height control overrides it.

Dense v3 query tiles contain activity and first/last values, not numeric extrema. At zoom-out these lanes show a labelled **dense activity · zoom in** band, which retains subpixel activity even when the endpoints agree. Zoom in for exact sample levels; the viewer does not present that band as a measured min/max amplitude envelope. No additional native protocol or simulator ABI is required.

7.4 Navigate the trace

Action	Result
Click / Shift-click	Place cursor A / cursor B
Left / Right, Change buttons	Previous / next active-signal change
Shift+Left / Shift+Right	Previous / next rising edge
Ctrl+Left / Ctrl+Right	Previous / next falling edge
Wheel	Zoom around the pointer
Shift+wheel	Pan horizontally
Left-button drag	Pan horizontally; a click without dragging places the cursor
Middle-button drag	Pan horizontally and scroll signal rows vertically
Drag the overview	Move the active time range
Fit, or double-click a viewport	Show the complete trace
A↔B	Zoom between cursors

Waveforms update while panning, without waiting for the mouse button to be released. The viewer fetches a bounded margin around the visible interval and coalesces requests while moving. If a large jump outruns the loaded data, the unfetched part stays empty until it arrives; the last value is not extended into unqueried history.

Without an active signal, navigation searches all displayed signals. Previous and Next exclude the current timestamp, including when a cursor lies between integer timestamps. Search also supports exact numeric values, bit patterns (`?` matches one bit), changes, rising/falling edges, and entry into X or U. Numeric values accept decimal, `0x` hex, and `0b` binary. Rising/falling searches apply to scalar signals.

Snap aligns a cursor with a nearby visible transition. The overview shows activity across the complete trace; drag its viewport rectangle to move through time. At dense zoom levels, activity buckets retain short pulses and U/X/Z activity. Zoom in to see exact transitions and values.

7.5 Values, bookmarks, and measurements

The signal list shows separate A/B values and a B−A delta. Cursor values are queried independently of the visible range. The Inspector reports transition counts, scalar rising/falling edges, high/low duration, and duty cycle. Duty is unavailable when an interval contains unknown or missing values. Known buses also have signed and unsigned deltas. U, X, and Z have distinct colors.

Native traces use `zlang_event_ticks`: two ticks represent one atomic simulation event. The Inspector shows event deltas and no physical frequency. External traces with a declared physical timescale also show seconds and `1 / Δt`. Traces without a physical timescale remain in trace ticks.

+ Bookmark records cursor A with a label. Click it to jump, double-click to rename, or use **×** to delete it. **◀ M / M ▶** move between bookmarks. Sessions support up to 256 bookmarks; colors can also be set in the session or through WCP.

7.6 Portable sessions

Run **ZLang HDL: Save Waveform Session** to write a `.zvw.json` sidecar, and **ZLang HDL: Load Waveform Session** to open its trace and restore the workspace. The trace path is relative to the sidecar where possible, so they can move together.

Save Session and **Load Session** are also available on the toolbar in both saved and live views. A live sidecar is an explicit layout template with `"source": {"kind": "live"}` instead of a `trace` path. Load it while a ZLang debug session is active. It restores the workspace against that session's recorded catalog, not a previous simulation's samples or state. Unrecorded or incompatible signals stay unresolved; add them to the live trace to rebind exact names and widths. The file does not execute simulation events or enable traces implicitly. Saved-file sidecars retain their existing relative `trace` path.

The `zlang-waveform-session-v1` document stores order, groups, dividers, presentation, pane sizes, viewports, synchronization, cursors, bookmarks, selection, and search settings. It stores no samples. VS Code workspace state provides automatic recovery separately.

When a catalog changes, rebinding requires exact hierarchical names and widths. Missing, ambiguous, or incompatible signals remain visible as unresolved rows. Saving again preserves those identities for later repair. Invalid sessions are rejected before replacing the layout. Sidecars are bounded to 1 MiB, 4096 signal rows including copies, 8192 total rows, eight group levels, and four viewports.

7.7 Live debugger traces

Start a native debug session, then run **ZLang HDL: Open Live Waveform**. The LIVE line marks the debugger event. **Follow Live** keeps it visible while stepping adds data. Clicking in the waveform disables following to inspect earlier data; the button enables it again.

An event file is optional for an interactive single-clock run. Follow the RuleAction manual-input walkthrough to add a launch with `clock: "clk"`, `cycles: 20`, and no `events` field. Press **F5** to stop on entry, then change `enable` under **Variables → Ports** using **Set Value**. **F10** completes one clock event and adds a waveform sample. An **F9** breakpoint on `when enable` or its state-update action stops **F5 / Continue** after an enabled event and highlights the selected source line. Editing an input alone does not advance the clock or add a trace sample. The example starts from the declared register value, not a reset sequence; use JSONL for explicit reset or repeatable input/clock sequences.

The Python simulation example also needs no JSONL, but creates a separate native instance. This live panel does not attach to it, and `.zhl` breakpoints do not stop that Python-driven instance. Use the ZLang debug launch for source breakpoints and live waves, or generate JSONL in Python and replay it in the debugger.

To trace a debugger value, right-click it in **Variables** or **Watch** and choose **Add to Waveform**. Aggregate fields, indexed values and tagged-union variants are supplied by the compiler's typed catalog. Interfaces appear as groups with independently directed fields. **Type** format shows enum members, signed/scaled values and explicit inactive/unknown-variant labels; raw radix formats remain available. See the typed tracing workflow for memory selection, limits and saved-file metadata boundaries.

The same Design Browser, duplicate rows, Cut/Copy/Paste, groups, 16-color palette, row heights, radices, search, edge navigation, A/B cursors, bookmarks, measurements and synchronized split viewports are available for live data. **Add signals** opens the debugger's trace selector; the browser lists signals already being recorded. Memory cells remain selectable through Variables. **Export Trace** writes the recorded raw backing signals to FST or VCD while the debug session is active. Open that file for inspection after ending the session; nominal live struct/union metadata is not embedded in the export.

Live layouts autosave per program/top/profile in the VS Code workspace. Closing and reopening the panel restores the layout, and restarting the same session keeps groups, styles, split views and bookmarks while clearing the old trace epoch. Cursor and bookmark times are clamped to the new available history. Clearing the displayed rows stays empty until signals are explicitly added. Changing to another debug design does not carry over the previous design's rows.

Moving cursors never changes simulator state. Step/Continue remain debugger actions. Signals added during a run have a history-start notice. Restart to record them from the beginning; restart clears the previous trace epoch.

Saved and live views request bounded render tiles, values, and measurements from Rust. Complete sample histories are not transferred to JavaScript. Files are limited to 1 GiB, render requests to 4096 signals and 8192 pixels, and responses to a width-dependent budget. Each document's response cache has a 128 MiB accounting ceiling; Rust also limits its decoded-signal cache to 128 MiB. File indexes and active render buffers are separate from these caches. Timestamps beyond JavaScript's exact integer range produce a diagnostic rather than silent rounding.

7.8 Troubleshooting

- **No waves in a saved file:** expand the Design Browser and click **+** beside a signal, then **Fit**. An empty signal list does not display every signal automatically. If simulation replaced the file, wait for it to finish and click **Reload**.
- **No waves while debugging:** check that a `zlang-hdl` session is active and open **ZLang HDL: Open Live Waveform**, not a saved FST/VCD tab. Use **Add signals** or **Variables → Add to Waveform**, then execute an event. A signal first enabled for tracing during a run has no earlier history; restart for history from event zero. Adding another row for an already recorded signal does not discard its history.
- **No color palette:** click a displayed signal row, open **Inspector**, and look under **Selection → Color**. Selecting only a browser entry or group does not open signal presentation controls. On a narrow window, Inspector opens over the right side. Both saved and live views offer the 16 colors, **Default**, and the numeric **Row height** control.
- **Refresh does not advance the LIVE marker: Refresh** only rereads recorded history. Use debugger **F10 / Step Over** or **F5 / Continue** to execute events. Moving cursors and changing input values while paused do not step the simulator either.
- **The installed viewer looks older:** run `make editor-install` from the checkout you are using, run **Developer: Reload Window**, and reopen the waveform. Start a new debug session for the matching compiler and adapter.

7.9 Waveform Control Protocol

WCP is disabled by default. Enable `zlang.waveform.wcp.enabled` in a trusted workspace; port `0` chooses an available loopback port. **Output → ZLang HDL** reports the address.

WCP v0 remains compatible. Negotiate v1 for `set_selection`, `set_cursors`, `set_bookmarks`, `export_session`, `import_session`, `set_viewport_sync`, and `get_measurements`. Unknown versions and commands return errors. WCP session export/import transfers the document; filesystem Save/Load remains an explicit editor action.

`set_selection` accepts row IDs for selecting an individual copy. Signal IDs or hierarchical names select all displayed copies of that signal. `focus_item` selects exactly one row by its row ID.

The workspace reads VCD and FST, including numeric analog-style lanes. GHW, expression evaluation, protocol decoding, source cross-probing, driver tracing, and simulator-state mutation remain outside its supported surface.

8 Generate SystemVerilog

Write the selected top to an explicit file:

```
mkdir -p build
zlang design.zhl --top Top --systemverilog build/Top.sv
```

The command writes only the requested artifact and keeps stdout empty. Diagnostics and `--verbose` progress messages go to stderr. A bare `zlang design.zhl --top Top` invocation writes SystemVerilog to stdout, which is convenient for a pipe but less suitable for a reproducible project script.

Use `--check` when no RTL should be created:

```
zlang design.zhl --top Top --check
```

Lint the generated module with Verilator:

```
verilator --lint-only --top-module Top build/Top.sv
```

Project profiles can select a target and clock constraint:

```
zlang top.zhl --profile release \
  --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc \
  --constraints-sdc build/Top.sdc
```

Generated RTL preserves declared widths, signedness, reset modes, hierarchy, and protocol behavior. Unsupported combinations stop with a diagnostic rather than emitting a partial module. Public port names are stable; private helper names are not a source-language interface and should not be referenced by user code.

Use `--systemverilog` for file output or the bare command for stdout.

8.1 Connect to vendor FPGA debug tools

Use `Tee<T>` or `RvTee<T>` from `std.stream.composition` to expose observation values through explicitly declared debug outputs. A direct wire assignment is also sufficient. Put the generated ZLang module inside your board-level HDL wrapper: its debug ports then become internal nets connected to Vivado ILA or selected in Quartus Signal Tap, without additional package pins.

The complete vendor example includes runnable ZLang/JSONL and AMD/Intel wrappers. Vivado/Quartus own debug-core generation, trigger/capture settings, programming and acquisition. Select a suitable sample clock and apply vendor preservation settings; a `tee` is only wiring and adds no CDC, storage or capture engine. Functional logical equivalence does not guarantee unchanged fanout, resource use or routed timing. Vendor project and board qualification are separate from native/Direct-SV passthrough checks.

9 Implementation profiles

Implementation profiles keep physical policy outside portable `.zhl` source. They are named tables in `zlang.toml` and are selected explicitly:

```
[profiles.release]
backend = "systemverilog"
backend-mode = "required"
target = "xc7z030ffg676-1"
allowed-transforms = ["pipeline", "dsp", "reduction"]
avoided-transforms = ["reassociate"]
objective = "maximize fmax"
architecture = "Xilinx7SymmetricDSPCascade"
architecture-mode = "preferred"
evidence-policy = "measured_required"
formal-policy = "required_bmc"

[profiles.release.constraints]
latency = { maximum = 8 }
ii = { exact = 1 }
fmax = { minimum = 100 }
dsp = { maximum = 8 }
```

`maximize fmax` prefers a legal choice with the strongest compatible frequency measurement available under the selected evidence policy. It does not guarantee the frequency achieved on an FPGA or ASIC. An estimate is never reported as measured routed timing.

Schematic use inside a project containing the shown `src/fir.zhl` and profile:

```
zlang src/fir.zhl --profile release --systemverilog build/Fir.sv
```

A profile may select pinned scalar external-module mappings with `external-mappings = ["vendor-add"]`. Mapping declarations live in the top-level `external-mappings` tables of `zlang.toml`; their source files and hashes are fixed by `zlang.lock`. They may change generated SystemVerilog but do not change the source-language behavior of the design.

Only the selected profile is parsed strictly. This permits a project to carry profiles for newer compilers without making unrelated builds fail. Selecting an unknown profile, selecting a profile without a project, or using an unknown key is an error. Profile data is excluded from dependency resolution and lock contents.

9.1 Semantic regions

The compiler reports one stable identity for each public scalar wire-output region. A profile can address an exact subset with `regions = ["DIGEST", ...]`. Generate `--implementation-policy-report` once to obtain the 64-character digests.

Region identity is derived from the logical module specialization, typed public output binding, canonical expression identity, and exact result type. Source paths, spans, physical instances, and generated RTL names are excluded. Stale and duplicate identities fail explicitly. Profile-selected regions are root-module-only and scalar-only; recursive protocol regions are unsupported.

9.2 Normalization and conflicts

`choice(auto)` remains accepted for user-supplied alternatives. Scalar `architecture(auto)`, `pipeline(auto)`, and `explore` are retired and produce migration diagnostics. Canonical `implement` source policy is represented through the same `ImplementationRequest` model as the selected profile and explicit compiler options. Equal normalized contributions are harmless. Different values for the same target, transform set, objective, constraint, evidence policy, formal policy, or architecture are rejected with both origins in the diagnostic.

For an ordinary typed scalar output, profile transforms/constraints/objectives run through bounded exploration. Canonical `implement` regions have already run during semantic analysis, so their retained candidate table is compared but never run a second time. Retained legacy records are handled the same way for replay only; no new transform or equivalence rule is introduced.

An exact module `timing` block is immutable public behavior. Profile bounds may equal or contain that exact latency/II, but cannot weaken or contradict it.

9.3 Reading the backend selection report

`--backend-implementation-report PATH` writes a JSON explanation of what the Direct-SV backend could actually emit for the requested physical implementation. This is **not** another source-language plan or an additional backend. Inspect it when a target/profile request does not produce the resource you expected.

Report status	Meaning
`selected`	Requested physical implementation was selected and emitted.
`generic_fallback`	`preferred` route was unavailable; generic RTL was emitted instead.
`unsupported`	Requested physical route is unsupported; `required` fails compilation.
`not_requested`	No physical implementation route was requested.

Current boundaries are deliberate: one backend is selected per named profile, II is limited to existing semantics, and profiles cannot introduce protocol, CDC, memory, retiming, or formal capabilities that the typed compiler does not already support.

10 Target platform descriptions

ZLang keeps functional behavior, implementation architecture, and physical target data separate. Explicit selection maps the ordinary functional symmetric FIR example to four DSP48E1 resources on `xc7z030ffg676-1`. Without an explicit selection, the same source continues through the generic Direct-SV path. Bounded automatic selection is described under target-aware planning.

10.1 Compiler-shipped source descriptions

Target data is ordinary compiler-shipped ZLang source:

- `std.target.generic` defines the resource-free generic target;
- `std.target.xilinx.series7` defines the bounded DSP48E1 capability;
- `std.target.xilinx.xc7z030` defines the device part, inventory, and cascade capacity;
- `std.arch.xilinx7_fir` defines the one manual symmetric-FIR template;
- `std.target.asic.sky130` describes the real `sky130_fd_sc_hd` standard-cell library boundary and proves that the core records are not FPGA- or Xilinx-specific.

The core parser and IR understand generic resources, typed ports, operation and width limits, register sites, dedicated links, inventories, targets, and architecture requirements. The names `DSP48E1`, `PCIN`, and `PCOUT`, and the physical SystemVerilog binding all originate in the library definitions - not in functional ZLang or semantic branches.

The SKY130 profile intentionally emits ordinary RTL for downstream Liberty-based mapping. It records the documented `dfxtp_1`, `dfrtp_1`, and `dlclkp_1` cell identities, but does not claim a hard multiplier, SRAM, PLL, fixed device inventory, timing closure, or silicon signoff. Those inputs are PDK/project/flow-specific and must be supplied by the implementation flow.

An exact source-level `timing { latency N ii 1 }` block is public module behavior, not target-selection policy. Generic implementation graphs publish their derived `timeless`/`known`/`unknown` timing class and are explicitly backend-independent. A selected physical graph carries its realization backend so a Direct-SV resource plan cannot be attributed to another physical implementation. Target selection must preserve any exact public module latency.

The bounded Series-7 profile describes a signed 25-bit pre-adder, signed 25x18 multiply, 43-bit product, 48-bit P/PCIN/PCOUT path, optional A/B/D/AD/M/C/P register sites, and an adjacent/same-group 48-bit dedicated cascade. The XC7Z030 instance records 400 DSP slices and four locally available cascade positions; this is deliberately not a complete floorplan model.

10.2 Manual selection

```
zlang examples/symmetric_fixed_fir.zhl \
  --target xc7z030ffg676-1 \
  --target-architecture Xilinx7SymmetricDSPCascade \
  --target-architecture-mode required \
  --systemverilog build/SymmetricFixedFIR.sv \
  --implementation-manifest build/SymmetricFixedFIR.manifest.json
```

`required` turns every target, shape, width, inventory, dedicated-link, or backend failure into a diagnostic. `preferred` returns the generic graph on a mapping failure. `generic` and omission of target options preserve the existing generic implementation. This command performs explicit selection; bounded automatic target-aware selection is documented separately in the Target-aware architecture and pipeline planning.

10.3 The selected graph

The matcher covers typed semantic structure only. It requires eight products, four coefficient expression identities reused at mirrored sample indices, and one final `nearest_even`/`saturate` conversion to `fixed<16,14>`. It does not use module names, source spelling, runtime coefficient equality, or test vectors.

For `SF2.10` samples and coefficients the legality evidence is:

Quantity	Required	DSP48E1 profile
pair pre-add	13 signed bits	25
coefficient/B	12 signed bits	18
exact product	25 signed bits	43
exact four-pair accumulation	27 signed bits	48

The resulting `ImplementationGraph` has four resource instances and three `DedicatedPhysicalEdge` records:

```
dsp0.PCOUT -> dsp1.PCIN -> dsp2.PCIN -> dsp3.PCIN
```

Each node maps the left/right sample expressions to A/D, the shared coefficient to B, and its predecessor/result identities to PCIN/P/PCOUT. Every internal DSP register site is explicitly zero in this configuration. The ordinary semantic output register provides latency 1; II is 1. The complete 48-bit cascade result then crosses the single original quantization boundary. No intermediate is narrowed.

10.4 Backend and manifest

The Direct-SV emitter consumes the selected graph; it does not rediscover a FIR. Its source-selected physical binding instantiates four Series-7 `DSP48E1` primitives with `INMODE=00100`, `OPMODE=0010101`, `USE_DPORT=TRUE`, direct A/B, and all selected internal register stages disabled. Actual `PCOUT` ports drive the next primitives' actual `PCIN` ports. A distinct behavioral resource model is used only for Verilator simulation.

The versioned implementation manifest retains semantic bindings and target/family/template identities, dependency hashes, selection policy, resource configuration, dedicated edges, latency/II, intended counts, and the emitted artifact hash. Intended resource use is not a vendor measurement.

10.5 Boundaries

The generic Direct-SV implementation remains available when no primitive graph is selected. Existing semantic-reference equivalence can validate the semantic fixed-point region, but its reference emitter does not model vendor primitives. Physical evidence therefore consists of the exact semantic oracle, Verilator execution of the separate resource behavior model, and real Vivado synthesis/place/route of the primitive artifact. No primitive-level formal infrastructure was added.

The automatic planner covers only its documented symmetric-FIR and signed-product regions; general placement/graph covering, DSP48E2, Intel physical emission, automatic storage/clock-resource mapping, and new formal machinery remain outside the current bounded support.

11 Target resources

This low-level layer describes legal resources and physical bindings; it does not itself select a design. The separate accepted high-level target-aware planner automatically selects the bounded symmetric-FIR and signed-product candidates documented there. Bounded ported-memory planning can match an exact advertised memory shape, replicate 1R1W resources for 1W+nR, or use a bounded register/mux fallback. Automatic banking and clock-resource planning remain unsupported.

11.1 Implemented model

Compiler-shipped ZLang source now defines generic, AMD 7-series, Intel Cyclone V, and ASIC-compatible resources. Resource definitions carry generic class, operation, capabilities, typed ports/limits, semantic pipeline sites, legal pipeline configurations, dedicated edges, and an opaque backend binding.

The loader validates duplicate/unknown pipeline sites, configuration latency, II, memory port/width/capacity, clock input/output/output-count requirements, dedicated edges, and target inventory. The same parser accepts project-local resource declarations; resolving project packages is deliberately deferred.

The generic library contains ordinary-RTL descriptions for logic, registers, multiplier/add/MAC, FIFO storage, RAM/ROM, clock/control abstractions, and carry. Selecting `generic`, or compiling without a target, leaves typed semantic IR and Direct-SV semantics unchanged.

11.2 DSP48E1 pipeline binding

The bounded symmetric FIR maps through source-described `DSP48E1` resources. Its semantic sites are:

```
input_preadd -> multiply -> accumulate_output
```

The physical binding maps them to A/B/D registers, MREG, and terminal PREG. Only configurations with compatible typed latency, II, and target capabilities are eligible. Routed measurements, if requested by policy, must match the exact physical graph and constraints. Separate target-resource validation records contain measured configurations; their QoR numbers are not portable language semantics or implicit guarantees for a different design.

11.3 Signed-product cascade

The same source-defined DSP48E1 binding now also covers the ordered `SignedProduct Reduction` graph used by the FFT scalar real and imaginary components. The Direct-SV emitter consumes the per-resource `accumulator_plus_product`/`accumulator_minus_product` configuration and emits ALUMODE controls without rewriting the finite-width expression. The final fixed-point conversion remains one typed boundary after the cascade.

Routed evidence, graph identities, and tool invocations belong in separate target-resource validation records. Evidence for another packed ABI, target, constraint, or graph is not reused.

11.4 RAMB36 validation

`examples/target_bram_memory.zhl` is an ordinary 1024x36, one-cycle, read-first synchronous memory with independent read/write addresses. Generic Direct-SV emits ordinary RAM RTL. The manual binding `Xilinx7BRAM36SimpleDualPort` consumes one RAMB36 capability and emits the same semantic template with block-memory physical binding.

The binding requires the resource's exact read-first collision contract; write-first bypass across independent addresses is not silently reinterpreted as a simple-dual-port primitive. Generic RTL may still be selected under an appropriate fallback policy. Resource inference measurements belong to target validation reports rather than the language contract.

Named true-dual selection is a distinct RTL shape. Generic same-clock multiport storage retains one deterministic process, but the selected Xilinx 2RW route requires `contents preserve` and

emits one physical process per port. A uniform `init VALUE` is preserved as a power-up/bitstream initialization under that reset policy. Native inference of a resource in a particular synthesis run does not upgrade a cross-clock collision model to an exact vendor guarantee.

The manually selected FIFO-only architectures `Xilinx7AsyncFifoRAMB18` and `Xilinx7AsyncFifoRAMB36` require the compiler-owned registered-read `async_fifo` decomposition, an independent-clock 1W1R port shape, exactly one read cycle and `DO_REG=0`. They do not select a public `async_mem`. Its physical plan owns ordered, width-aware pointer, Gray, full, ready/valid and prefetch equations. The Direct-SV emitter renders those equations and the digital simulator evaluates the same equations. Separate target-resource validation records hold synthesis observations; physical clock/reset and metastability requirements remain implementation responsibilities.

The live resource capability schema now distinguishes `simple_dual`, same-clock `true_dual`, and independent-clock `1W1R`; it also separates same-clock from cross-clock collision guarantees. A legacy `true_dual` flag is not sufficient evidence for an asynchronous collision contract. The current 7-Series data intentionally advertises no exact cross-clock old/new guarantee, so strict target-required `async_mem` publication fails closed while generic Direct-SV remains an explicitly labelled structural digital model.

11.5 Intel and ASIC status

Cyclone V source data describes variable-precision multiplier modes, 64-bit accumulation, chain connectivity, generic pipeline sites/configurations, M10K port/width modes, ALM/FF distinctions, and a fractional PLL resource. The same generic legality checks used for RAMB/MMCM validate M10K/PLL requests.

Intel primitive emission is unsupported: a selected Intel physical binding fails closed with `unsupported`. The current release publishes no Quartus-backed synthesis or QoR validation for this target. The compiler-shipped `sky130_fd_sc_hd` profile identifies the real high-density standard-cell library plus its documented positive-edge DFF, active-low-reset DFF, and integrated clock-gate cells. Direct-SV remains generic RTL for downstream Liberty mapping. SKY130 SRAMs, PLLs, die-area inventory, PVT corner, timing closure, and physical signoff are deliberately not fabricated by the compiler; project macro libraries and the ASIC implementation flow own them.

Emit technology-neutral RTL carrying the SKY130 target identity with:

```
zlang design.zhl --top Top --target sky130-fd-sc-hd \
  --systemverilog build/Top.sv
```

The downstream synthesis/P&R invocation must still supply the selected `sky130_fd_sc_hd` Liberty, LEF and process configuration.

11.6 Clock-resource blocker

The target libraries can describe clock input/output ranges, output counts, phase capability, and lock/reset metadata. Current functional ZLang clock domains do not express a frequency/phase relationship from which a truthful PLL/MMCM configuration could be derived. Physical clock emission therefore remains unsupported: there is no source-level frequency/phase requirement from which to select a configuration. Raw PLL generics are not exposed in normal modules.

11.7 Implementation artifact contents

Implementation manifests now retain selected pipeline configuration, active semantic sites, physical-binding identities, and separate intended, emitted, and measured resource-count slots plus timing provenance. Existing semantic bindings remain independent. Current compiler-produced artifacts populate intended/emitted counts after successful physical emission; external vendor tools own measured counts and routed timing.

Automatic target-aware planning is enabled only for the reviewed symmetric-FIR and ordered signed-product reduction regions documented in the Target-aware architecture and pipeline

planning. General `explore`, BRAM/clock selection, and arbitrary resource ranking remain disabled. The accepted boundary is documented in the Target-aware architecture and pipeline planning.

12 Target-aware architecture and pipeline planning

The bounded planner is implemented for two exact fixed-point region families on `xc7z030ffg676-1`: the symmetric FIR cascade and ordered signed-product reductions used by complex multiply (`p0 - p1` and `p0 + p1`). It extends the existing architecture alternatives/exact reduction planning/pipeline scheduling/timing alignment/deterministic cost selection/bounded exploration path; it is not a second exploration engine.

12.1 User contract

```
result = implement {
    quantize<fixed<16,14>>(acc) {
        round nearest_even
        overflow saturate
    }
    intent { latency <= 8 ii == 1 fmax >= 100 }
}
```

`ii` and the compatibility spelling `throughput` normalize to one canonical II constraint. Fmax values are MHz. `latency<=N` is a bound. `latency==N` is an exact observable sample latency. `maximize fmax` is an evidence-ranked selection objective, not an achieved-frequency guarantee.

The target is build configuration, for example:

```
zlang examples/symmetric_fixed_fir_auto.zhl \
  --top SymmetricFixedFIRAuto \
  --target xc7z030ffg676-1 \
  --target-evidence-policy measured_required \
  --systemverilog build/SymmetricFixedFIRAuto.sv \
  --implementation-manifest build/SymmetricFixedFIRAuto.json \
  --pipeline-report build/SymmetricFixedFIRAuto.report
```

Functional source never names DSP48E1, MREG or PREG. With no target, or with `--target generic`, the ordinary generic candidate and backends remain active.

12.2 Candidate generation and legality

The symmetric-FIR region is exactly an eight-tap signed fixed reduction where four coefficient semantic values are reused at mirrored sample indices. Runtime coefficient equality is not symmetry. Eight independent coefficients therefore remain generic. The signed-product region is derived from the already typed `SignedProductReduction`: ordered product terms and each add/subtract join are preserved, and the resource must advertise the required accumulator modes. Arbitrary subtraction trees or reassociation are not inferred.

The candidate set is:

1. generic implementation;
2. source-described symmetric cascade with no resource-local site;
3. multiply site selected;
4. multiply and terminal output sites selected;
5. input/pre-add, multiply and terminal output sites selected.

These are the four configurations published by the resource library. Core code does not enumerate primitive register bits. The signed-product family applies the same configuration names to its exact two-resource chain. Before cost extraction the planner validates the typed pattern, exact pre-add/product/accumulator widths, required add/sub capability, inventory, dedicated edges, II, and latency.

The final `FixedConvert` stays after the full 27-bit/F20 accumulation. A mapping requiring hidden narrowing is rejected and the generic candidate remains.

12.3 Timing DAG

Every selected target graph carries a backend-independent timing DAG with stable semantic/implementation identities. It records resource combinational segments, selected resource-local cuts, the three dedicated cascade edges, fabric boundaries, final fixed quantization and output boundary.

Timing alignment produces explicit `alignment` delay objects. Exact external latency produces separate `compensation` objects. Both record cycles, width and fabric FF cost in the graph and manifest. They are never reconstructed from RTL names.

For the selected useful latency-three implementation:

- `latency<=8` adds no delay;
- `latency==8` adds one explicit five-cycle output compensation delay;
- resource-local useful sites remain unchanged.

The Direct-SV emitter consumes this graph. The base output boundary and five compensation cycles form the required output register chain; source `pipeline(N)` latency is not added again.

12.4 How target evidence affects selection

This topic belongs here only as a practical distinction: an estimated resource cost is not a measured FPGA result, and an estimated Fmax is not routed timing.

Evidence kind	What it establishes
`structural_estimate`	Compiler estimate from the selected structure; no vendor run.
`synthesis_measurement`	Result after synthesis, not placed/routed timing closure.
`routed_measurement`	Measured result after implementation/routing for matching target and constraints.

`measured_required` accepts only compatible routed evidence for an Fmax constraint. Structural latency/II remain authoritative and are not erased by that policy. Compatibility includes target/-part, architecture, complete graph hash, pipeline configuration, backend, clock constraint and tool/version. `measured_preferred` ranks the strongest compatible evidence cohort before the numeric objective and falls back to estimates only when no legal stronger candidate remains. This prevents a coarse structural Fmax estimate from being compared as though it were an equally authoritative routed measurement.

Packaged routed evidence is bound to the matching part, constraints, physical graph, pipeline configuration, backend and tool recipe; a different graph's measurement cannot be reused as proof of this one. The optional `--implementation-manifest PATH` records the selected configuration, resource and timing decisions and their provenance for inspection or replay. Internal JSON schema numbers are compatibility details, not language features or knobs users should set.

12.5 Current boundary

Automatic target planning is intentionally limited to the symmetric-FIR and signed-product Direct-SV regions above. BRAM, PLL/MMCM, Intel physical planning, arbitrary graph covering, II-changing sharing, and automatic fixed transformations are not enabled. Generic Direct-SV remains the fallback implementation. Existing formal infrastructure is unchanged and no primitive-level proof claim is made.

13 Platform constraints

13.1 Clock constraints

A selected project profile can declare one explicitly named clock period:

```
[profiles.release.platform.clocks.clk]
period-ns = 10.0
```

The clock name must match the selected top's clock declaration. The period is a physical build setting and is not inferred from an `fmax` intent.

The CLI writes constraints only together with exactly one SystemVerilog output. The following command assumes a project-local `top.zhl` and the `release` profile shown above:

```
zlang top.zhl --profile release --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc --constraints-sdc build/Top.sdc
```

XDC and SDC initially contain only:

```
create_clock -name clk -period 10 [get_ports {clk}]
```

The clock edge and reset mode must match the source declaration. A stale or mismatched constraint request is rejected. XDC/SDC output is deterministic and is written atomically without following symlinks.

No reset false path is emitted: asynchronous-reset recovery/removal timing must not be hidden. This constraint publication supports only the exact single-clock profile shown here. Multiple clock domains are supported in language semantics and Direct-SV emission, but generated clocks, multi-clock XDC/SDC publication, pin/package/IO properties, input/output delays, CDC constraints, and formal properties are not emitted by this facility.

14 Direct-SV

The backend consumes typed semantic IR. It never dispatches on AXI, APB, or RegBus module names: source-authored standard-bus components use the same aggregate endpoint, hierarchy, register, rule, and connection lowering as user modules.

Ordinary scalar register/rule components share the standalone/composed dispatch and register contributor. Independent rules do not require global scheduler enumeration merely because a module is instantiated as a child. Hierarchy validation is cached only within one emission; protocol/storage effects and conflicting multi-effect rules keep their existing exact paths. Generated private names now use a shared hierarchy-local allocation plan: `cfg_ready`, `lane_0`, `lane_0_value`, `result_pipe_s1`, and short specialized definitions such as `Counter_s1a2b3c4d`. Source names win over generated helpers; actual collisions receive a deterministic suffix. Public top/port ABI and source register names are unchanged, including existing public aggregate-leaf separators. BackendArtifact records the naming schema and complete semantic identities separately from regenerated RTL/VPI locators. Naming does not change circuit semantics or imply QoR gains.

Formal-only `rule.fire` now uses the same effective reset and polarity as production state, including the two-edge synchronized release and conditioned child reset. Real RTL covers both polarities, both active edges and nested hierarchy. Its separate global-guard enumeration scalability limitation remains: large independent-rule formal projections remain outside the scalable supported subset.

Fixed-point ports and expressions use the canonical scaled-integer types behind `fixed`/`ufixed`, concise `SF`/`UF`, and `_Sat` formats. Same-scale target narrowing and explicit rescaling are both emitted from `FixedConvert`; the backend never infers wrap, saturation, or rounding from source spelling.

14.1 Supported composition subset

- scalar and packed struct/vector/tuple datapaths, `char`/`string<N>` aliases, immutable locals, muxes, field/projection access, range-proven runtime packed-bit indexing/fixed-width packed slices and runtime vector indexing with LSB-zero semantics, exact-width `bitcast`, homogeneous vector `concat`, compile-time `reshape`, reductions, fixed-point arithmetic/conversion, and selected or nested fixed-latency scalar pipelines with II=1;
- registers, next-state assignments, atomic rules, FSM lowering, reset, and priority in one or more explicitly owned domains, including one independent two-register asynchronous-assert/synchronous-release conditioner per applicable domain;
- reusable parameter specializations, distinct scalar physical child instances, and compile-time indexed combinational or single-domain sequential instance arrays, including aggregate scalar outputs and scheduled FIFO state combined with ordinary child registers/rules;
- mixed scalar/state/ready-valid/request-response children;
- mixed scalar plus ready/valid instance-array children, bounded nested same-domain scalar/direct-ready-valid array hierarchy, and first-level indexed in-order request/response requester/responder arrays;
- read-only runtime projection of one exact bit-packable scalar/aggregate wire output from an instance array; every physical child remains independently instantiated and active;
- the bounded single-input/single-output ready/valid `transform pipeline(auto)` with one pure pipeline scheduling product-reduction kernel and a compiler-owned global stall;
- ready/valid connections and finite connection FIFOs, including simultaneous push/pop and backpressure;
- scalar ready/valid `async_fifo(N)` clock crossings and aggregate crossings whose schema has exactly one forward ready/valid member; aggregate payloads cross as one packed atomic value;
- vendor-neutral scalar `sync_level`, `pulse_toggle`, and one-entry ready/valid `handshake` crossings, using the same coordinated-reset and latency semantics as the simulator;
- synchronous memories with the currently typed collision/read-latency model;
- fixed-priority and round-robin packet arbiters with beat- or packet-scoped grants;
- in-order hierarchical request/response channels with independent directional buffers as represented in typed IR;
- one mandatory public top ABI derived from `TopPhysicalABI`: structs are recursively ex-

posed as named field leaves, tuples as deterministic `itemN` leaves, while every `vec<N,T>` leaf is a multidimensional packed SystemVerilog array (`logic [N-1:0][W-1:0] name` for a `W`-bit element); flat packed aggregate representations exist only as deterministic aliases inside the selected top or inside compact child components;
- source-authored AXI4-Lite, APB, AXI-Stream, Wishbone, RegBus, and CSR hierarchy used by the current standard-library and real-design examples;
- hardware-connected CSR `ro`, `pulse`, and sticky-W1C behavior with the frozen software/hardware priority policy;
- ready/valid-to-credit, credit-to-ready/valid, and per-VC credit source state machines from typed protocol/capacity metadata;
- recursive instance/state locators and exact physical-domain contracts for non-default reset modes. Specialization identity is not used as physical instance identity.

SystemVerilog reserved words are deterministically prefixed with `zlang_`. The collision check runs after that physical-name mapping, so two distinct semantic leaves can never silently become one RTL port. Inline-boundary helper signals are allocated outside the public namespace and remain deterministic. Packed dimensions use conventional descending ranges. ZLang element zero maps directly to physical packed index zero and the least-significant private slice. The generated artifact has one unconditional top definition and is accepted by both Yosys and Verilator without backend preprocessor branches. Unresolved locals are eliminated before backend lowering. Unsupported IR shapes raise `SystemVerilogEmissionError`; no artifact is published after a failed emission.

Ordered comparisons preserve typed signedness explicitly: both operands of <, <=, >, and >= are rendered through `$signed` or `$unsigned` according to typed IR, independent of whether an operand is a port, local, projection, or register. Equality and inequality remain raw bit comparisons.

Use the stable CLI option:

```
zlang examples/simple_dma.zhl --top SimpleDMA \
  --systemverilog build/SimpleDMA.sv
verilator --lint-only --top-module SimpleDMA build/SimpleDMA.sv
```

`--systemverilog` is an explicit artifact sink: it writes only the requested SV file and leaves stdout empty. A bare `zlang SOURCE` invocation emits the production Direct-SV artifact to stdout. Diagnostics remain on stderr.

Use `--verbose` when an explicit success confirmation is useful:

```
zlang design.zhl --systemverilog design.sv --verbose
```

This keeps stdout artifact-safe and reports the selected top and written paths on stderr. To validate parsing, top selection, and semantic analysis without publishing any backend artifact, use:

```
zlang design.zhl --check
```

`--check` prints a one-line success result and returns zero. Without `--top` it checks every module declared in the source file, preventing an invalid dormant module from being hidden by default-top selection. With `--top`, it checks only that selected elaboration root. Syntax, semantic, and top-selection failures retain their normal nonzero status and diagnostic. As a check-only mode, it cannot be combined with artifact output options.

14.2 Simulation-only architectural state access

A Direct-SV build may publish a separate Verilator VPI companion without adding ports or changing the production RTL:

```
zlang design.zhl \
  --systemverilog build/design.sv \
  --simulation-state-bundle build/state-access
```

The companion manifest covers the selected typed hierarchy and is bound to the exact emitted artifact hash, build identity, state shapes, and emitter-owned physical locators. Publication rehashes the `.sv` file, so a stale or modified RTL file cannot receive an access manifest. The generated C++ header requires Verilator `--vpi --public-flat-rw` and validates its complete state allow-list before access.

The bounded surface includes bit-packable user registers (including vector registers), writable-memory cells, and the persistent read-result latch of a one-cycle memory. Values up to 64 bits have convenience methods; arbitrary-width scalars and elements use exact 32-bit least-significant-word-first arrays. Vector element zero retains the canonical least-significant packed position. The persistent semantic simulator consumes the same catalog and keeps one state object per physical child instance.

14.3 Formal boundary

Real Z3/SymbiYosys regression targets exercise emitted direct RTL for register transitions and rule priority, ready/valid stability and FIFO accounting, CSR W1C behavior, and request/response acceptance. Arithmetic, state-transition, FIFO-accounting, response-accounting, W1C, and priority mutations all fail with counterexample metadata.

The first-class verification-bundle path also accepts Direct-SV designs with initialized ROM. Exact memory images are published below `implementation/companions/` and listed as hash-validated inputs for every SBY job that consumes them. Generated solver configuration, logs, and VCDs are retained outside the immutable bundle; the run report maps sampled physical VCD signals back to semantic binding IDs for witnesses and counterexamples.

Direct-SV is the executable bundle route. If its formal emission cannot bind a required observation, the goal is explicitly skipped or rejected according to policy. Unsupported aggregate/protocol shapes report a non-executable reason; no generated-name reconstruction or alternate backend substitution is used.

`RegBusCSRTarget` stores response data and holds `response.valid` until `response.transfer`. Source-authored AXI-Lite and APB frontends can therefore complete write transactions through the typed RegBus boundary.

The implementation artifact publishes recursive physical locators only for signals actually present in emitted RTL. Current Direct-SV formal emission publishes the typed accepted-request, directional-buffer occupancy, response-consumption, and parent-owned outstanding-ledger observations used by the supported recursive request/response properties. Register, FIFO, CSR, and rule-fire observations are likewise executable where their complete typed binding sets exist. A `formal_observation_token` remains absent for hidden or unsupported state, and the affected property reports `skipped` rather than fabricating a proof.

14.4 Explicitly unsupported

Legacy globally controlled storage combined with user state, out-of-order or nested request/response arrays, additional protocols on a request/response array child, arbitrary nested storage/CSR/aggregate protocol hierarchy, CDC arrays, and runtime-selected inputs, protocols, or actions remain fail-closed. The backend never silently emits only one element or reconstructs an indexed connection from an RTL name.

Sequential ready/valid modules and role-qualified standalone in-order `request_response` requester/responder modules use the same typed clock/reset, ownership, and accounting semantics as their hierarchical forms. A standalone endpoint owns a local unbuffered ledger: request transfer increments it, response transfer decrements it, and reset starts a new empty epoch. Stateful generic child templates that still require parent specialization remain validated through their concrete parents.

14.5 Example and structural coverage

The release checks every standalone-supported example root through Direct-SV emission and strict Verilator lint. Child/template roots require a concrete specialization or parent. These tests are evidence for the shipped examples, not a language allow-list; unsupported new combinations fail closed. Current corpus counts and tool versions are published in release status.

The streaming FFT and IEEE 802.11a transmitter examples exercise typed hierarchy, ROMs, fixed-point datapaths, protocols and state. Their RTL is built through the same generic backend path as other designs, without module-name dispatch. Selected examples also have independent cycle-level oracle checks.

Generated RTL uses exact packed widths and single-driver combinational assignments. Procedural combinational blocks are used where defaults or branching require them; strict lint treats width, latch, driver and structural warnings as failures. A target binding is claimed only when the selected physical capability proves its clock, latency, collision and reset contract.

15 Formal verification

ZLang HDL supports named same-cycle safety properties and bounded cover goals. Place goals in the source next to the hardware they describe:

```
module CheckedCounter {
    clock clk reset rst
    in enable : bit
    out value : u8

    reg count : u8 = 0
    step: when enable & (count < 10) { count <- truncate<8>(count + 1) }
    value = count

    assert count_is_bounded @ clk { count <= 10 }
    cover reaches_ten @ clk { count == 10 }
}
```

Run all applicable goals for a selected top:

```
zlang design.zhl --top CheckedCounter --verify \
  --formal-jobs 4 \
  --verification-report build/verification.txt \
  --verification-work-dir build/verification-work
```

For a reproducible two-step flow, first publish a verification bundle and then run it:

```
zlang design.zhl --top CheckedCounter \
  --verification-bundle build/verify
zlang verify build/verify --mode bmc --depth 20 \
  --work-dir build/verify-work
```

The work directory contains solver logs and any counterexample or cover VCD. Keep it outside the bundle so reruns do not modify the input being checked.

Result terms have precise meanings:

- `bounded_pass`: no counterexample was found through the stated depth; this is not an unbounded proof;
- `proven`: the selected proof flow completed an unbounded proof;
- `witnessed`: a cover goal was reached and has a witness;
- `bounded_unreached`: a cover was not reached within the selected bound;
- `unknown`, `unavailable`, or `skipped`: no proof claim was established.

Each clocked goal names its clock with `@ clock_name`. Reset behavior follows the module's declared polarity, assertion mode, and release contract. Goals in independent supported domains run separately; this does not create a cross-domain temporal property. Use explicit assumptions only for real environment constraints, and never use an assumption to hide an invalid design.

For a source checkout, `make venv PYTHON_PROFILE=dev` and the default release profile include the pinned Python Z3 package. A runtime-only checkout can add Python bindings with `python -m pip install -e '.[formal]'` after activating its local environment. This extra contains no test or release tools. It enables optional arithmetic proof preparation and does not install Yosys or SymbiYosys. Without these bindings the existing raw SMT route remains available; its result must still satisfy the requested proof policy. The Python package pin and the actual external solver identity are checked separately.

Formal execution requires Yosys, SymbiYosys, `yosys-smtbmc`, and a supported solver such as Z3. The installation chapter shows how to check those tools. Examples are available in `examples/verification`.

16 Whole-build manifests and evidence

ZLang can describe one reproducible build as a deterministic, versioned record. The whole-build manifest joins the compiler stages, selected implementation policy, backend products, companion files, actual tool executions, generated reports, and typed evidence without making any of those records the source of language semantics.

The first schema is `zlang-whole-build-manifest-v1`. It is complementary to, not a replacement for, the per-backend `BackendArtifact` manifest.

16.1 Compiler and implementation identities

A build records two distinct canonical compiler identities:

- `high-level:<sha256>` identifies the fully typed, backend-independent high-level canonical IR;
- `selected:<sha256>` identifies the selected canonical architecture IR.

The identities use the versioned canonical-identity schema and exclude source spans and other attribution-only metadata. An optional canonical content hash may accompany either reference. The high-level and selected identities are not backend artifacts, RTL hashes, or synthesis-plan identities.

The production Direct-SV backend is planned from the selected-IR identity. Its record keeps these identities separate:

- the normalized backend plan identity;
- the backend `build_identity`;
- the exact emitted artifact hash and BackendArtifact version;
- an optional physical implementation-graph identity;
- an optional source-map hash.

This distinction prevents a physical Direct-SV resource plan from being mistaken for the selected semantic design. Backend states are explicit: `selected`, `generic_fallback`, `unsupported`, `failed`, or `not_requested`. A selected or fallback build must publish an artifact, its manifest version, and at least one content-addressed output. Unsupported, failed, and unrequested plans cannot silently publish successful products.

16.2 Published files and companions

The manifest records the exact compiled root-source bytes, the complete locked project and compiler-shipped `std.*` dependency closure, backend outputs, and backend companion bundles as `PublishedFile` values. Every value contains:

- a normalized relative POSIX path;
- a SHA-256 content hash;
- a file kind;
- the exact byte size for build products;
- optional ZLang source attribution.

Absolute paths, `..`, non-normal paths, and duplicate publication paths are rejected. Directory-root validation additionally rejects symlink escapes. The CLI's explicit logical-path-to-physical-`Path` map checks every build product's type, size, and content hash before publishing the manifest. Locked dependency records are the exception: the project workspace has already validated their content hashes before semantic analysis, so their physical cache/check-out paths are intentionally absent from the public manifest validation map.

Before any product is written, the CLI also checks a private, non-semantic map of every physical compiler input: the root snapshot, project manifest, lock, root and dependency modules, dependency manifests, and every consulted stdlib candidate. Explicit file sinks, generated companions, and compiler-owned output or cache directories may not alias or contain any of those inputs. File sinks may not alias one another, and owned directories may not overlap. Physical paths protect the workspace from overwrite; they never enter semantic, artifact, cache, or whole-build identity.

Companions remain associated with the backend product that needs them. For example, an initialized ROM image is not hidden inside the RTL identity: the image is a separate content-addressed companion validated beside Direct-SV `$readmemb` output.

The CLI acquires the root source once as raw UTF-8 bytes. Semantic compilation and manifest hashing use that same snapshot; CRLF is preserved, and a physical source change before final publication rejects the manifest. Generated RTL and ROM companions are published through one relative-path publisher: every directory and leaf is opened without following symlinks, the payload is fsynced in a same-directory exclusive temporary, renamed atomically, and revalidated by content. Symlinked or non-regular destinations fail without writing through them. Every published companion retains one semantic identity and is hash-validated independently.

Generated source maps and rendered reports are content-validated in the same way. A report's stable semantic identity is derived from its report ID, kind, and ordered evidence IDs. Its rendering format, content hash, and optional publication path remain serialized and tamper-checked, but intentionally do not alter the whole-build identity. A report cannot refer to unknown evidence, and a published report path cannot carry different bytes.

16.3 Exact evidence meanings

Evidence is adapted only from typed compiler/formal result objects. Log strings and ad-hoc result dictionaries are not evidence inputs. The stable statuses mean:

Status	Exact meaning
`typed_legal`	Semantic analysis produced a well-typed module. This is not a timing or proof result.
`timing_validated`	Every public scalar output was checked against the exact typed module timing contract. This is not a proof result.
`bounded_pass`	BMC found no counterexample through the recorded positive `depth`. It requires mode `bmc` and is never described as unbounded proof.
`proven`	An unbounded `prove` execution succeeded. Only this status denotes an unbounded proof.
`failed`	The executed check found a failure; typed counterexample attribution may be attached.
`unknown`	Execution completed without a pass, proof, or counterexample classification.
`skipped`	The route was explicitly unavailable or inapplicable. It is not success.
`not_run`	No proof execution occurred. It is not success.

`bounded_pass` must include BMC mode and depth. `proven` must include prove mode. Semantic and timing validation cannot carry proof mode/depth. An unexecuted record is rejected if its claim says that something was proved.

The typed adapters preserve the applicable property ID, candidate identity, backend and artifact hash, reference hash, engine, solver, relation, proof route, mode, and depth:

Positive formal-aware selection evidence requires a connected backend and artifact. The report layer does not infer execution from cache metadata, registered rewrite rules, generated properties, an SBY file, or a solver executable being installed.

16.4 Harness generation is not proof execution

`--formal-harness` and `--formal-sby` generate formal inputs. They do not run a solver and therefore cannot produce `bounded_pass` or `proven`. When no typed execution result exists, the corresponding property is reported as `not_run` and may explain why execution was not performed, but it must not imply success. Artifact generation itself belongs in the backend/publication records, not in a second evidence status.

The first-class verification UX adds an immutable bundle boundary rather than changing that rule. `--verification-bundle` publishes hash-validated structured verification IR,

implementation/source-map inputs, and separate safety/cover jobs. Exact ROM images are immutable `companion` inputs to every job that uses them. `zlang verify` or `zlang --verify` creates a run report only after real execution. Solver, engine, depth, timeout, logs, tool versions, witnesses, and counterexamples are run data and are not folded into the bundle's source identity. A bounded cover miss is `bounded_unreached`, never proof of unreachability.

The bundle describes immutable inputs. A separate run report records `run_identity`, exact execution configuration, retained bounded-stage results, tool versions, and a work directory for each safety/cover job. Generated SBY configuration, stdout/stderr, timeout diagnostics, and VCDs live outside the bundle below `--work-dir`/`--verification-work-dir`. A retained VCD may be decoded through the immutable binding table into semantic signal/value pairs; paths and raw logs remain outside `run_identity`.

A joint `--verify` plus non-`off` formal-policy run publishes a compiler report that links the raw execution report to the exact compiler execution plan and separately typed selected-candidate semantic-reference equivalence reports. Candidate reports retain deterministic per-route work roots and the discovered tool snapshot when execution needed tool discovery; exact in-session formal-aware selection reuse also carries its recorded work root. Physical paths remain operational metadata outside every semantic, run, evidence, and cache identity. Persistent formal-aware selection cache payloads omit them rather than claiming that an old workspace is still available.

Each safety/cover job is tied to one exact clock/reset pair. Multiple supported synchronous domains may therefore appear as independent jobs in one bundle; an unsupported domain skips only goals that name it. This is not a cross-domain equivalence or temporal proof relation.

A requested prove run is staged behind BMC. Proof starts only when every safety job is `bounded_pass`. Covers run once during BMC and are not rerun; an unrelated cover result does not block safety proof, while an unwitnessed feasibility cover first makes its dependent safety result vacuous/unknown and therefore blocks proof. The merged report retains both the bounded cover evidence and any proved safety results. This sequencing is part of execution, not bundle identity.

A safety counterexample or an executed joint semantic-reference equivalence counterexample is a verification failure. Missing/unknown/vacuous safety verification/source evidence or an unsatisfied requested proof is incomplete. A bounded cover miss is non-failing, and unavailable advisory candidate evidence neither changes formal-aware selection eligibility nor makes an otherwise complete joint run incomplete.

Likewise, merely detecting Verilator, Yosys, SymbiYosys, or Z3 does not create a tool-execution record. A tool record represents an actual normalized invocation and includes its role, exact reported version, path-free command shape, result status and exit code, logical outputs, and proof mode/depth when applicable. Tool output paths must resolve to files already published by the build.

16.5 Counterexample and source attribution

Source origins remain attribution, not semantics: they include the logical source unit, source digest, span, and construct, survive manifest JSON round-trips, and are excluded from deterministic evidence and build identities. Moving an unchanged locked project therefore does not change its build identity.

16.6 CLI publication

The intended publication forms are:

```
zlang design.zhl --systemverilog build/design.sv \
  --evidence-report build/evidence.json \
  --evidence-format json \
  --build-manifest build/zlang-build.json
```

`--evidence-format text|json` selects deterministic human-readable or JSON evidence. `--evidence-report` publishes that report. `--build-manifest` publishes the whole-build join after requested outputs have been written and their hashes validated. The manifest is fsynced to a same-directory temporary, renamed atomically, its physical publications are validated

again, and the parent directory is fsynced; a detected intervening output mutation removes the manifest. A whole-build manifest requires a real backend product; report-only or check-only commands do not fabricate a successful build.

The build identity includes locked source/dependency hashes, both canonical IR identities, normalized implementation request/policy and optional profile, independent backend records, actual tool executions, reports, evidence, and stable metadata. Ordering is canonical. Host paths, timestamps, wall time, cache hit order, Python insertion order, and other volatile runtime state are not part of it.

16.7 Current limitations

These boundaries preserve the existing formal-infrastructure freeze while making build and evidence claims reproducible, attributable, and mechanically auditable.

17 Structured diagnostics

Every compiler error retains its human-readable message and may also carry a stable diagnostic code, one primary `SourceOrigin`, notes, and suggested fixes. `SourceOrigin` always has a half-open source span and construct; its logical source unit and lowercase SHA-256 digest are optional in the base record. A *complete* origin has all four fields. Compiler-shipped declarations use logical units such as `std.math.complex`; ordinary CLI inputs use the path supplied to `zlang`.

Text remains the default and is compatible with existing scripts:

```
zlang design.zhl --check
```

The `fixes` values in the structured diagnostic are human-readable suggestions, not source edits. Machine-applicable edits have a separate compiler-owned metadata and tooling projection described in compiler-owned diagnostic edits; consumers must never derive edits from these strings.

Machine consumers select one deterministic JSON object:

```
zlang design.zhl --check --diagnostic-format json
```

The JSON object has `schema`, `severity`, `code`, `message`, `primary`, `notes`, and `fixes`. Older exception-string users continue to receive the same `str(error)`. Categories initially carrying specific codes include parsing, imports, width assignment, timing alignment, domain crossing, protocol ownership/type, top selection, I/O, and backend binding failures. Unmigrated errors use a stable generic category rather than inferring semantic meaning from diagnostic text.

Backend output can additionally publish a hash-bound generated source map. External-tool attribution is accepted only when the generated text hash matches the sidecar and exactly one entry covers the reported line. Otherwise the original Verilator/Yosys/vendor diagnostic is left unchanged.

18 Generated source maps

ZLang backends can publish a deterministic JSON sidecar alongside an immutable `Backend Artifact`. The sidecar is backend-independent: each published entry connects an inclusive generated line range and semantic identity to a complete typed `SourceOrigin` (source unit, digest, span, and construct). The base origin record permits an absent source unit or digest, but such an incomplete record cannot establish fresh cross-file attribution. The sidecar also records the backend, module, selected-IR identity, and generated artifact SHA-256, so a map cannot silently be applied to different generated text.

The sidecar format is implemented by `zlang.backend.source_map.GeneratedSourceMap`. Direct-SV offers `emit_artifact_with_source_map`; this returns the unchanged production artifact plus its sidecar model. `write_sidecar` writes canonical, sorted JSON. The CLI exposes the same model for one explicit production output:

```
zlang design.zhl --systemverilog Design.sv \
  --source-map Design.sv.zmap.json
```

The sidecar artifact hash must match the generated file before a consumer uses any line attribution.

For published consumers, the versioned validated generated-navigation bundle now provides the missing explicit file/source-snapshot integrity boundary. It does not expand this map's bounded line coverage and does not add an LSP navigation method.

The external-tool helper applies the same rule to Verilator messages: it adds a `ZLang origin:` line only for one exact mapped generated line after hash validation. Changed files, malformed locations, ambiguous entries, and unmapped helper/state-machine lines retain the original tool diagnostic unchanged.

18.1 Exactness boundary

The current generated-source-map implementation maps only a simple top-level output assignment when all of the following are true:

- the typed output expression retains a complete `SourceOrigin`;
- the `BackendArtifact` publishes the matching semantic output binding;
- the emitter's generated assignment statement is uniquely identifiable; and
- the generated text hashes to the artifact identity.

Direct-SV maps a unique `assign <published-token> = ...` statement. Ambiguous, sequential, multi-output, hierarchical, protocol, and backend-generated helper lines remain deliberately unmapped. No mapping is inferred from similar source and RTL names. Later emitter refactoring may attach exact origins while fragments are constructed; until then, absence of an entry means “unknown,” not “same as the nearest mapped line.”

18.2 Generated-RTL navigation boundary

A generated source map gives exact generated **line** ranges, not columns. Only a unique top-level output assignment with a retained complete compiler `SourceOrigin` is currently mapped; the final output assignment of an eligible pipeline may also qualify. Module declarations, internal state, helpers, protocol bridges and hierarchy are otherwise unmapped. An origin's span and construct are always present; its logical source unit and digest are optional in the base record, but a complete mapped origin has all four.

A consumer must first verify the generated text against the artifact SHA-256. Zero matching entries means “unmapped”; multiple distinct complete origins are ambiguous and must not be guessed from names or proximity. The bundle can validate source-snapshot freshness, but there is no supported source-to-generated query or LSP navigation method. Generated coordinates are one-based lines; source spans use compiler-native code-point columns, converted to UTF-16 only by an LSP client boundary. See the generated-navigation bundle for publication and integrity details.

19 Generated navigation bundles

ZLang can publish a small, relocatable Community artifact bundle that binds one already-generated Direct-SV file to its exact generated source map, backend manifest, and producing source snapshots. The bundle is an integrity and provenance boundary for tooling. **It does not provide an LSP navigation method.**

Publish it during the normal compiler invocation:

```
zlang design.zhl --top Design \
  --systemverilog build/Design.sv \
  --source-map build/Design.source-map.json \
  --build-manifest build/Design.build.json \
  --generated-navigation-bundle build/Design.navigation
```

The source-map and whole-build outputs remain independently optional. The bundle always contains its own exact generated file, generated source map, and serialized `BackendArtifact`; it never invokes a second backend render.

19.1 Why generated navigation uses a separate bundle

Before this contract, `BackendArtifact`, `BackendBuildRecord`, and `WholeBuildManifest` already carried backend/module/selected-IR identities, artifact hashes, dependency identities, and relocatable logical product paths. `GeneratedSourceMap` carried the same artifact lineage and exact mapped source origins. Those records did not provide a single persisted path from which a consumer could locate the arbitrarily placed physical RTL and sidecar, and the whole-build root publication did not retain the compiler logical source unit.

The navigation bundle is a narrow publication contract that avoids changing unrelated whole-build consumers. It reuses the canonical serialized `BackendArtifact` as the lineage owner instead of duplicating its hierarchy, implementation, timing, module-signature, and dependency structures.

19.2 Layout and schema

All persisted paths are normalized relative paths below the bundle root:

```
manifest.json
generated/design.sv
generated/source-map.json
manifest/backend-artifact.json
```

`manifest.json` identifies its own format for strict compatibility checks. It contains:

- `bundle_identity`: SHA-256 of the canonical identity payload;
- `generated_artifact`: a `PublishedFile` logical path, SHA-256, kind, and size;
- `source_map`: the same record for canonical `GeneratedSourceMap` JSON;
- `backend_manifest`: the same record for canonical `BackendArtifact` JSON;
- `sources`: an ordered list of `{role, source_unit, digest}` records;
- `schema` and `schema_version`.

Exactly one source has role `root`. Every compiler-owned dependency from the backend artifact's locked dependency closure and resolved library dependency set is recorded with role `dependency`. Absolute checkout/build paths are not serialized. Package/revision identities remain in the hashed backend manifest; source snapshot matching uses the exact logical source unit plus SHA-256.

The root record is mandatory even when `GeneratedSourceMap.entries` is empty. No fake source-map entry is created. Consequently an empty map can still report the root source as matching or stale.

19.3 Publication and validation

`publish_generated_navigation_bundle()` accepts an already-emitted `BackendArtifact`, its `GeneratedSourceMap`, and the already-written generated file. It verifies the physical generated bytes against `artifact_hash`, checks backend/module/selected-IR/map lineage, derives source snapshots from existing compiler dependency metadata, and publishes the four relative files with the existing no-follow atomic publication utility.

`load_generated_navigation_bundle(path)` starts from one explicit bundle directory. It does not scan for `.sv` or JSON files and does not infer filenames from a module. Loading validates:

- bundle schema and canonical identity;
- safe relative paths and no-follow containment;
- presence, regular-file status, exact sizes, and SHA-256 of all child files;
- canonical backend-manifest and source-map serialization;
- generated bytes against the backend artifact hash;
- Direct-SV backend, module, selected-IR, and source-map lineage;
- exact source snapshot agreement with backend dependency provenance;
- complete `source_unit` and digest identity for every mapped source origin.

Any mismatch is an error. Missing files, stale/tampered contents, sidecars from another artifact, unsupported schemas, path traversal, and symlinked bundle components fail closed. Loading performs only metadata parsing, file reads, and SHA-256 checks; it never parses ZLang, compiles, emits RTL, runs formal tools, or starts an external process.

19.4 Source freshness

The immutable loaded record provides two protocol-neutral comparisons:

- `source_status(source_unit, current_text)` hashes current UTF-8 text;
- `source_digest_status(source_unit, current_digest)` compares a supplied SHA-256 directly.

Both return exactly `match`, `stale`, or `unknown_source`. Unknown sources are never treated as matching. Bundle facts are validated at load time; current editor text is checked separately because it can change after loading.

19.5 Relocatability and limitations

The complete directory can be moved or copied and loaded at its new location. No original absolute path is required. Current bundles are Direct-SV only and do not embed producing source contents. They validate caller-supplied source text/digests but do not locate workspaces or manage unsaved overlays.

Source-map coverage remains the bounded line-only coverage documented in Generated source maps. generated-navigation bundle adds no generated columns, recursive line mappings, virtual documents, artifact index, implicit compilation, or source-to-generated query. **There is no LSP generated-RTL navigation yet.**

20 Compiler tooling API

ZLang HDL exposes a small, read-only Python integration surface in `zlang.tooling`. It is intended for editors, build coordinators and other compiler-aware tools that must not depend on parser, workspace or semantic IR implementation modules.

Consumers must check `TOOLING_API_SCHEMA`; the package version alone is not a compatibility guarantee. The API publishes immutable records for:

- compiler, source-suffix and capability identity;
- source declarations and direct-import resolution;
- the immutable document-symbol projection for the current source text;
- the immutable semantic-hover projection for one source position;
- the immutable compiler-resolved definition projection for one source position;
- the immutable compiler-resolved reference projection for one source position;
- the immutable semantic rename-edit projection for one source position;
- the immutable compiler-visible semantic completion projection for one source position;
- the immutable compiler-resolved signature-help projection for one source position;
- the immutable compiler-classified semantic-token projection for one source;
- compiler-owned machine-applicable diagnostic fixes for semantic checks;
- read-only project/workspace indexing and dependency closure;
- one exact semantic-only source-snapshot check.

These calls do not emit RTL, run formal tools, synthesize a design or mutate a project lock. A failed semantic check is returned as a structured record. An environment or malformed-project failure that prevents a trustworthy record raises `ToolingError`.

20.1 Request-scoped analysis

Tooling queries request only the compiler observations needed for that query; they do not emit RTL or perform formal execution. A bounded compilation session may reuse analysis products after validating the exact project and editor snapshot. Definition, References, rename and semantic tokens use the same compiler-owned identity and occurrence records; no result is inferred from matching source text. See Language Server Protocol and VS Code for editor behavior and the API description above for integration details.

20.2 Diagnostic edits

`TOOLING_DIAGNOSTIC_EDIT_SCHEMA` is currently `1`. A failed `check_snapshot()` record retains the existing prose-only `ToolingDiagnostic.fixes` and may additionally contain immutable `ToolingDiagnosticFix` records. Each machine fix preserves its compiler-owned title and one atomic tuple of `ToolingDiagnosticEdit` records containing the absolute current-source path, exact compiler origin and replacement text.

Only explicit `DiagnosticError.machine_fixes` metadata is projected. Tooling does not parse diagnostic messages, codes, notes or legacy fix prose. The projection requires every edit to match the exact snapshot digest and a valid half-open span; otherwise the entire fix is omitted. The first and currently only producer deletes the later of two imports with identical logical path and alias. Cross-file machine edits are not projected in this schema.

The compiler/tooling records remain protocol neutral. They are not LSP `TextEdit`, `WorkspaceEdit`, `CodeAction` or `Command` values. The LSP layer leaves this schema unchanged and mechanically maps current-source machine fixes to edit-only LSP quick fixes after rechecking the current editor snapshot.

20.3 Generated RTL artifacts and navigation limits

`GeneratedSourceMap` v1 provides hash-verified generated-line to source-origin attribution, but its bounded builder currently maps only one unique top-level output assignment. It has no generated artifact path or artifact-discovery contract, and no query currently validates a map against the current editor source digest. Recursive `BackendArtifact` bindings describe semantic/physical signals and hierarchy, not generated line locations.

Consequently `zlang.tooling` does not expose raw `BackendArtifact` or source-map objects and does not generate RTL on a tooling request. The exact capability and prerequisite are recorded in Generated source maps.

The public `load_generated_navigation_bundle()` API validates one explicit relocatable publication directory, exact generated/backend-manifest/source-map hashes, lineage, and complete producing source snapshot identities. The returned immutable record can classify current source text or digest as `match`, `stale`, or `unknown_source`. This is artifact infrastructure only: `zlang.tooling` exposes no generated-location query yet, and loading never invokes compilation.

20.4 Document symbols

`TOOLING_DOCUMENT_SYMBOL_SCHEMA` is currently `1`. The `document_symbols(source_text)` function returns a tuple of immutable `ToolingSymbol` records. Each record contains a declaration name, a stable compiler-owned category, an authoritative `ToolingOrigin` where one is available, an authoritative selection range (currently the same origin when the parser does not retain a narrower name span), and recursively projected children where the parser exposes containment.

This is deliberately a narrow document-structure projection. It does not export AST nodes, typed IR, references, definitions, widths, protocols, or implementation plans. It uses the existing parser through the tooling module; the LSP must not parse or scan source text independently. If parsing fails for an incomplete editor buffer, `document_symbols` returns an empty tuple rather than guessing declarations. The existing `TOOLING_API_SCHEMA` remains `1` because this is an additive, separately versioned projection. Parser records that carry no stable declaration identity are omitted rather than assigned a synthetic symbol name.

20.5 Semantic hover

`TOOLING_HOVER_SCHEMA` is currently `1`. The `hover_at(source, source_text, line, character)` function accepts a stable source path and a zero-based editor position, then returns either an immutable `ToolingHover` record or `None`. The record contains only narrow compiler-owned facts: selected name and kind, canonical type text, width, signedness, fixed-point type text, port direction, an available callable signature, and its authoritative `ToolingOrigin`.

The function invokes the semantic-only compiler snapshot path. It does not invoke optimization, implementation selection, RTL generation, synthesis, or formal execution, and it never returns AST or typed-IR objects. Positions with no semantic entity and malformed/incomplete source return `None`; workspace, resolver, and other environment failures raise `ToolingError` so callers cannot mistake an unavailable project for a valid empty hover.

20.6 Definitions

`TOOLING_DEFINITION_SCHEMA` is currently `1`. The `definition_at(source, source_text, line, character)` function asks the semantic compiler snapshot for its resolved source-definition records and returns either an immutable `ToolingDefinition` or `None`. A definition contains only the resolved name/kind, the locked physical target path, and the authoritative target `ToolingOrigin`.

The projection currently covers compiler-resolved value/port references and ordinary or generic function calls, named types, enum members, registers, immutable locals, and module-instance targets when their declaration origin is retained. Function calls and declarations use parser-owned exact name spans. Imported project and stdlib definitions use the existing workspace lock and physical-input closure to map their logical source unit to the physical `.zhl` path. Unsupported declaration categories return `None` rather than being reconstructed from names or source text. Resolver and workspace failures remain `ToolingError`; the projection never exports AST, resolver, or typed-IR objects.

20.7 References

`TOOLING_REFERENCE_SCHEMA` is currently `1`. The `references_at(source, source_text, line, character, include_declaration)` function reuses the compiler's occurrence-to-target resolution records and returns immutable `ToolingReference` records containing only a locked physical source path and authoritative occurrence origin. Results are grouped by the compiler-owned

declaration origin, never by identifier spelling, and are sorted deterministically with exact duplicate locations removed.

When `include_declaration` is true, the authoritative target declaration is included. When false, only resolved usages are returned. Malformed or unresolved source returns an empty tuple; workspace and source-path failures remain `ToolingError`. For a saved declaration in a locked project, the tooling layer considers only project roots whose dependency closure can contain that declaration. A parser-only name prefilter bounds the expensive work, then each candidate root is independently checked and only its compiler-owned occurrence-to-target records may become results. Reported spelling is validated against the exact source span to reject malformed inherited provenance. The prefilter is not a reference search and cannot create a result. Unsaved declaration snapshots remain local so in-memory and disk identities are not mixed. The persistent normalized shard described below only replays these compiler-owned records; no spelling-based result fallback, resolver object, AST, or typed-IR object is exposed.

20.8 Rename

`TOOLING_RENAME_SCHEMA` is currently `1`. The `rename_at(source, source_text, line, character, new_name)` function resolves the target through the compiler-owned definition/reference identity and returns an immutable tuple of `ToolingRenameEdit` records, or `None` when the category is not rename-safe. Each edit contains a locked local source path, an exact parser-owned identifier `ToolingOrigin`, and the replacement text.

New names are validated with the parser's ordinary binding-name rule. Before returning edits, the tooling layer applies them to the current root snapshot and performs a semantic-only recheck; invalid names, collisions, retargeted references, missing exact spans, and cross-file edits fail closed with `ToolingRenameError`. The initial supported categories are local ports, immutable values, functions, and function parameters. Registers, instances, types, protocol members, and cross-file rename are intentionally unsupported. The LSP converts these records to the standard `WorkspaceEdit.changes` form; the tooling API does not expose AST, IR, resolver objects, or LSP classes.

20.9 Completion

`TOOLING_COMPLETION_SCHEMA` is currently `1`. The `completion_at(source, source_text, line, character)` function consumes compiler-recorded semantic scope snapshots and returns immutable `ToolingCompletion` records. The projection currently covers visible ports, immutable values, function parameters, ordinary/generic functions, and functions imported through the locked workspace. It does not reconstruct scopes, scan source text, add imports, or provide keyword/snippet completion.

Completion uses only the semantic check product. Malformed or incomplete source and positions outside an analyzed expression return an empty tuple; workspace and resolver failures remain `ToolingError`. The API does not expose the semantic scope, AST, resolver, or typed-IR objects themselves.

20.10 Signature help

`TOOLING_SIGNATURE_HELP_SCHEMA` is currently `1`. The `signature_help_at(source, source_text, line, character)` function returns either an immutable `ToolingSignatureHelp` record or `None`. The record contains one compiler-resolved callable label, its parameter labels, the active parameter index, and the authoritative call origin.

The semantic analyzer records a call only after ordinary, generic, or locked imported function resolution succeeds. Argument origins are retained by the compiler and are used to select the active parameter; the tooling and LSP layers do not parse commas, parentheses, or identifier text. Nested calls are resolved by choosing the most-specific compiler call span. Malformed or incomplete source and positions outside a resolved call return `None`, while workspace/environment failures raise `ToolingError`. Signature help demands only the semantic snapshot and does not run optimization, implementation selection, RTL generation, synthesis, formal tools, or external processes.

The projection deliberately exposes no AST, typed IR, resolver state, generic substitution machinery, or implementation metadata. Trigger characters are not advertised by the initial LSP

slice because requests are handled identically regardless of how they are triggered.

20.11 Semantic tokens

`TOOLING_SEMANTIC_TOKEN_SCHEMA` is currently `1`. The `semantic_tokens(source, source_text)` function returns immutable `ToolingSemanticToken` records for exact compiler-owned declaration and resolved-occurrence spans in the current root document. Tooling categories are protocol independent: `function`, `parameter`, `property` for ports, and `variable` for immutable values. Declarations carry the `declaration` modifier; usages do not.

The projection consumes the same semantic-only snapshot and exact identifier origins used by definition, references, and rename. It rejects broad or multiline spans, filters imported declarations out of the root document, orders records deterministically, removes exact duplicates, and fails closed on overlap. It does not classify source text, expose a lexer/AST/IR object, or assign numeric LSP legend indices. Malformed or semantically invalid source returns an empty tuple; project/environment failures remain `ToolingError`.

20.12 Reusable tooling sessions

A `ToolingSession` can reuse immutable semantic and symbol results for repeated editor requests. Reuse requires matching compiler/schema, selected root and profile, exact editor text, and the content of the locked dependency closure. Unsaved buffers stay in memory; saved project symbols may be cached on disk. Neither a matching name nor a timestamp alone is validity evidence. Corrupt or stale entries are ignored and checked again by the compiler.

The cache is an optimization, not an alternative language analysis. It never stores source bodies or generated RTL and does not grant unsupported navigation or change diagnostics. The user-facing cache switch is described under Cache configuration.

21 Language Server Protocol and VS Code

`zlang-lsp` is the single Community ZLang HDL language server. Its protocol surface provides deterministic diagnostics, document symbols, semantic hover, compiler-resolved definition locations, semantic references, safe semantic rename, deterministic semantic completion, and compiler-resolved signature help, full-document semantic tokens, and compiler-owned diagnostic quick fixes. It keeps open document text in memory and queries the compiler-owned tooling projections after full-document opens and changes.

Start it as a standard LSP process:

```
zlang lsp
```

`zlang-lsp` remains an installed compatibility executable for editor settings that accept a program path but cannot express subcommand arguments.

The process reads and writes standard Content-Length framed JSON-RPC messages. It supports local `file://` URIs for existing `.zhl` files. The server does not write editor buffers to temporary source files; project-backed snapshots must therefore remain compatible with the existing compiler/tooling workspace identity rules. Unsupported remote, virtual, and non-local document URIs are reported explicitly.

21.1 Supported now

- `initialize` / `initialized`;
- `shutdown` / `exit`;
- `textDocument/didOpen`;
- `textDocument/didChange` using full-text synchronization;
- `textDocument/didClose`;
- `textDocument/publishDiagnostics`;
- `textDocument/documentSymbol` for an open document;
- `textDocument/hover` for an open document;
- `textDocument/definition` for an open document;
- `textDocument/references` for an open document;
- `textDocument/rename` for the conservative rename-safe subset of an open document.
- `textDocument/completion` for compiler-visible expression-scope candidates.
- `textDocument/signatureHelp` for one compiler-resolved callable at the requested position.
- `textDocument/semanticTokens/full` for exact compiler-classified identifiers.
- `textDocument/codeAction` for compiler-owned machine-applicable quick fixes.

Diagnostics use the Community `zlang.tooling.check_snapshot()` API, which uses the normal parser, resolver, and semantic checker. They keep the compiler code, message, source origin, and one-based-to-zero-based range conversion. Document symbols use the narrow parser-owned `zlang.tooling.document_symbols()` projection over the current unsaved editor text. Hover uses the narrow semantic `zlang.tooling.hover_at()` projection over the same current text and source path. The LSP does not contain a second parser, type checker, workspace model, backend, formal runner, or optimizer.

Editor observations are demand-driven through one compiler-owned `AnalysisNeeds` mask. Definition, references, rename, and semantic-token requests collect only shared definition records; completion additionally requests scope candidates, and signature help requests resolved-call records. A definition request therefore does not build completion scopes or callable detail strings, while semantic checking and the resulting IR remain unchanged.

Document symbols are structural declarations, not a semantic database. Their names, categories, containment, and available source spans come from the compiler's parser representation. Incomplete or malformed text returns an empty symbol list while diagnostics continue to report the parse error; there is no regex or source-text fallback. A declaration whose AST node has no dedicated source origin uses the enclosing authoritative declaration span.

Hover reports only compiler-owned facts that are already available from the semantic IR: the selected name/kind, canonical type, width, signedness, fixed-point representation, and (for ports)

direction. Function declarations may include their compiler-owned signature. It does not run optimization, implementation selection, RTL generation, synthesis, or formal tools. A malformed/incomplete buffer or a position without a semantic entity returns `null`; workspace/environment failures remain explicit protocol errors.

Definitions use `zlang.tooling.definition_at()` and the compiler's actual name/import resolution. The server does not search source text, rebuild scopes, or choose same-named declarations. Project imports are mapped through the locked workspace to the target source URI; unresolved or unsupported categories return `null`.

For a file declaring several modules, Definition and References share one bounded navigation-context selector. A missing default-top occurrence is retried under the module preceding the cursor according to parser-owned module declaration spans. Only that selected module is then semantically analyzed; the compiler's exact occurrence-to-declaration record still decides whether F12 returns a location. A bounded session-local, exact-text selector avoids rechecking an unrelated generic last module on repeated F12; no source AST or physical path is added to persistent symbol shards.

Protocol connection endpoints retain exact parser-owned name spans. Each name is resolved by semantic elaboration, so a connection such as `command -> packet_mapper.command` can navigate independently to the parent port declaration, the instance declaration, and the selected child port. The cache stores only those compiler-produced relations.

VS Code may issue F12 at the exclusive right edge of its selected identifier. Tooling performs the exact half-open-span lookup first, then retries the immediately preceding code point only when the cursor is at an identifier boundary. It does not walk across whitespace or punctuation. Declaration paths outside the root project, including stdlib targets, are returned only when they are members of the compiler-owned locked physical input closure.

Definition coverage includes compiler-resolved module instance targets, named user types (including nested generic components, structs and aliases), nominal enum types, enum members, and local registers. Targets retain their locked source identity, digest, and parser-owned declaration span, so a dependency file need not be open in the editor. Builtin constructors such as `rv` and `bits` have no fabricated source definition. Source-local target/resource declarations also retain exact lexer-token name spans: `provides ResourceName` and `require_resource ResourceName` navigate to one unambiguous `resource ResourceName` declaration and participate in semantic References. Unknown or duplicate source-local names return no location. This does not yet implement general cross-file target-catalog navigation; physical target legality remains with the separate target planner. The server converts VS Code's incoming UTF-16 character offsets to compiler code-point columns before semantic span containment; outbound locations use the inverse conversion.

The extension completes language-client registration before advertising navigation support, so an immediate first F12 request is supported. It does not perform extension-side symbol lookup.

Generic declarations are semantically checked for their concrete specializations. When a generic-only library file is opened directly, the compiler's dedicated `ZL-GENERIC-SPECIALIZATION-REQUIRED` condition is not published as an editor error. A parameter constraint referencing a declared value parameter without a default is likewise deferred only while directly viewing its unspecialized template; concrete false constraints and unrelated runtime references remain editor errors. The production compiler still rejects an unspecialized top, and parse/concrete-specialization errors remain ordinary diagnostics.

A saved project child opened directly is checked using the semantic analyzer's existing child-boundary mode. This avoids treating an enum-bearing internal port as an illegal public top ABI merely because its parent is not the active editor document. The production compiler's default top-level boundary remains fail-closed; all other child semantics and concrete imported types are checked.

References use `zlang.tooling.references_at()` and the same compiler-owned occurrence-to-target relation. Results are grouped by resolved declaration identity, not by spelling, and may include the declaration when the LSP `includeDeclaration` flag is true. For a saved locked-project snapshot, the tooling layer builds an on-demand bounded view of dependency-compatible project roots and accepts only compiler-resolved occurrences with an exact source span. The project sweep reloads the current manifest/lock and reuses each exact root text for parser filtering and

semantic checking. It verifies disk bytes before publication; a concurrent edit or more than 128 eligible roots/top pairs reports an error instead of a stale or silently truncated result. A declaration above several modules in one source also checks bounded sibling tops that may use it; this includes encoded-enum uses inside a non-default module. A parser prefilter limits semantic checks but never produces a reference result. Qualified enum owners and members retain separate identifier spans, and incoming LSP UTF-16 cursor columns are converted before compiler lookup. The normalized symbol cache described below reuses compiler-produced shards; it does not create results. There is no text/regular-expression fallback, and an unsaved declaration snapshot is never mixed with disk roots.

Rename uses `zlang.tooling.rename_at()` and the same compiler-owned identity and occurrence records. It edits only exact parser-owned identifier spans, always including the declaration, and rechecks the edited in-memory root source with the semantic checker to reject collisions or retargeted references. Rename supports local ports, immutable values, functions, and function parameters. Cross-file edits and declarations without exact editable spans are rejected; there is no textual or same-spelling fallback.

Completion uses `zlang.tooling.completion_at()` and semantic scope snapshots recorded by the compiler while checking the current source. It offers only visible ports, immutable values, parameters, ordinary/generic functions, and locked imported functions. Candidates are deterministic, insert the plain identifier, and carry optional type/signature detail. The server does not reconstruct scopes, scan source text, synthesize imports, or provide keyword or snippet completion. Positions outside a compiler-analyzed expression and malformed/incomplete buffers return an empty list.

Signature help uses `zlang.tooling.signature_help_at()` and call records captured while the compiler resolves ordinary, generic, and locked imported function calls. The callable label, parameter labels, return type, call span, and argument spans are compiler-owned. The server selects the most-specific resolved call containing the cursor and determines `activeParameter` from those spans; it never counts commas or parses call text itself. A cursor in malformed/incomplete source, or outside a resolved call, returns `null`. No trigger characters are advertised until parser behavior makes a trigger contract useful.

Semantic tokens use `zlang.tooling.semantic_tokens()` and only the exact identifier spans already retained by compiler definition/reference records. The initial categories are functions, parameters, ports, and immutable values; declarations carry the standard `declaration` modifier and resolved usages do not. The server publishes a fixed standard-token legend, sorts tokens by source position, converts compiler code-point columns to the default LSP UTF-16 code units, and emits standard relative full-document encoding. It does not lex, scan, classify keywords, infer token roles from spelling, or expose hardware resource information. Malformed source produces an empty token data array.

Code actions recheck the exact current in-memory source through `zlang.tooling.check_snapshot()` and map only `ToolingDiagnostic.machine_fixes`. The current action is the compiler-owned deletion of a later completely identical duplicate import. Actions are `quickfix` edit-only `CodeAction` values, filtered by the request range, `context.diagnostics`, and `context.only`; stale diagnostics and fixes are not returned. Compiler-native spans are converted to LSP UTF-16 positions without changing replacement text. Prose-only diagnostic suggestions never become executable actions.

21.2 Cache configuration

The LSP reuses exact validated semantic and symbol results, including after reopening an unchanged saved project file. Changes to editor text, selected module, manifest, lock or any used dependency invalidate affected results; unsaved buffers are memory-only. A cache miss runs normal semantic analysis. Set `ZLANG_LSP_SYMBOL_CACHE=memory` to disable disk caching or `off` to disable symbol shards; `persistent` is the default. Cache corruption or a read-only cache location cannot turn a valid source into a navigation failure.

Existing source maps are hash-bound and exact for their few mapped generated lines, but they do not yet provide complete artifact discovery, current-editor-source staleness validation, or broad RTL coverage. The LSP does not load raw backend manifests, infer mappings from generated names, or generate RTL during navigation. See Generated source maps.

The generated-navigation bundle provides a protocol-neutral validated generated-navigation bundle for explicit published artifact discovery, integrity, lineage, and source-snapshot freshness. No LSP method consumes the bundle yet, and advertised server capabilities are unchanged.

21.3 Not implemented

`codeAction/resolve`, fix-all/source/refactor actions, semantic-token range/delta requests, workspace symbols, generated-RTL navigation, formal/synthesis commands, and other unlisted capabilities are outside the current public scope.

Source links target the public release tag `v0.1.0a24`. For a candidate, these web links become available when that tag is published. Links between the three manuals work offline.

Font license notice

Bitstream Vera / DejaVu fonts

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is a trademark of Bitstream, Inc. DejaVu changes are in public domain.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license (”Fonts”) and associated documentation files (the ”Font Software”), to reproduce and distribute the Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words ”Bitstream” or the word ”Vera”.

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the ”Bitstream Vera” names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED ”AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome Foundation, and Bitstream Inc., shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from the Gnome Foundation or Bitstream Inc., respectively. For further information, contact: fonts at gnome dot org.